



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения
(ИППО)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1-7

по дисциплине

«Методологии разработки программного обеспечения»

Выполнил: студент группы ИКБО-33-22

Шило Ю. С.

Проверил: преподаватель

Иерусалимов И.Д.

Практическая работа выполнена «__» _____ 2025 г.

(подпись студента)

Зачтено «__» _____ 2025 г.

(подпись преподавателя)

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА 1

Задачи:

1. Настройка проекта в системе Taiga.io:

- Создайте проект в Taiga.io и назовите его, исходя из особенностей проекта.
- Добавьте участников команды (разработчиков, тестировщиков и продуктового владельца и т. д.).
- Проанализировав предметную область, создайте список пользовательских историй (user stories) на основе требований к программному обеспечению.

Для этих целей, можно воспользоваться такими методами, как анализ конкурентных решений, анализ сценариев использования, интервьюирование стейкхолдеров и т. д.

2. Планирование спринта:

- Разделите user stories на задачи и определите их приоритеты.
- Оценить каждую задачу в часах или story points.
- Выберите задачи, которые будут включены в следующий спринт.
- Распределите задачи между участниками команды, сформированных ранее.

3. Создание и реализация спринта:

- Продумайте над порядком выполнения задач и реализации user stories.
- Занесите выбранные user stories и разбитые на задачи в Sprint Backlog в Taiga.io.
- Установите цель Sprint'a и его продолжительность.
- Смоделируйте выполнение данного спринта. Обращайте внимание на соответствующие метрики.
- «Проведите» ежедневные совещания для обсуждения прогресса и проблем команды. (Для решения данной задачи можно смоделировать ежедневные встречи).
- Обновить статус задач в Taiga.io и обозначьте возможные сложности

препятствия, используя соответствующие возможности Taiga.io.

- Смоделируйте проведение ретроспективы спринта для обсуждения прошлых достижений и улучшения процесса.
- Занесите обнаруженные улучшения в план действий на следующий Sprint.
- Смоделируйте проведение демонстрации завершённой функциональности.

4. Формирование следующих спринтов:

- Сформировать ещё несколько спринтов и определить в них требуемые пользовательские истории.
- Повторить процесс, который описан в пункте 3, с начала, начав новый Sprint с обновленным Product Backlog и новыми целями.

Результат работы:

Создадим Scrum проект для веб-платформы “Онлайн Аптека”. (Рисунок 1).

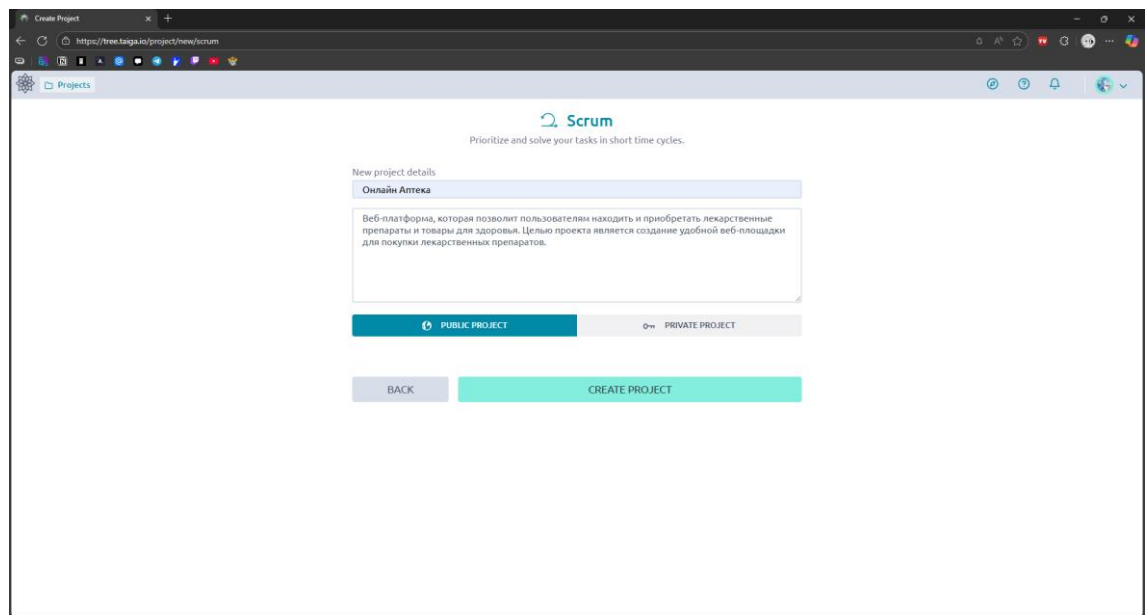


Рисунок 1 – Создание Scrum-проекта

Добавим дополнительных участников в проект. Для этого перейдем в Настройки-> Участники -> новый участник. Также необходимо выбрать роль нового участника (Рисунок 2).

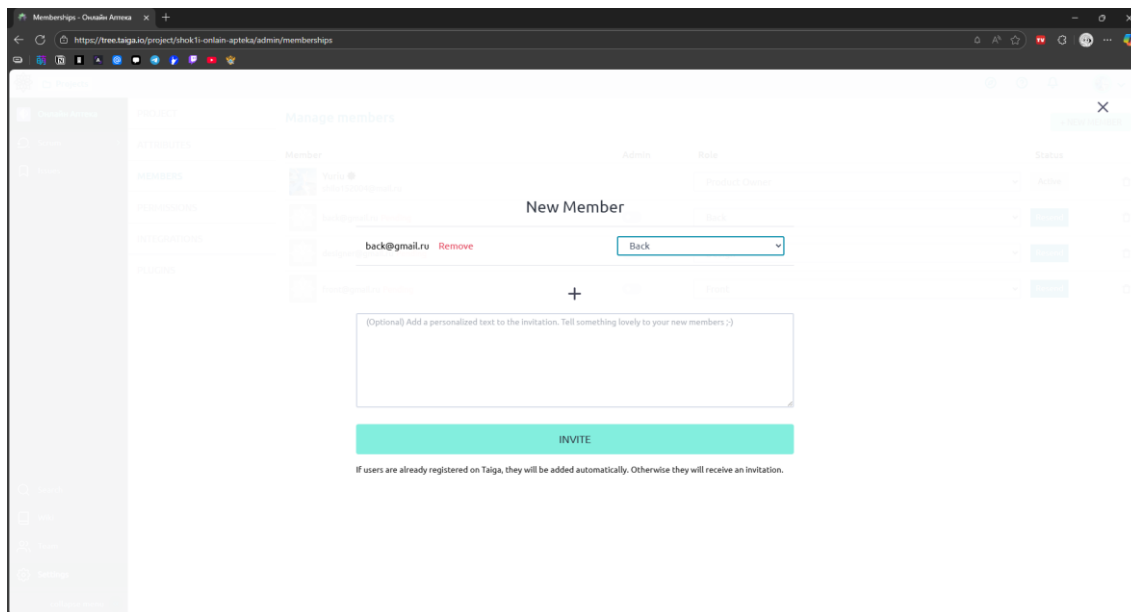


Рисунок 2 – Добавление нового участника

По аналогии добавим разработчика игровой логики, итого получим (Рисунок 3).

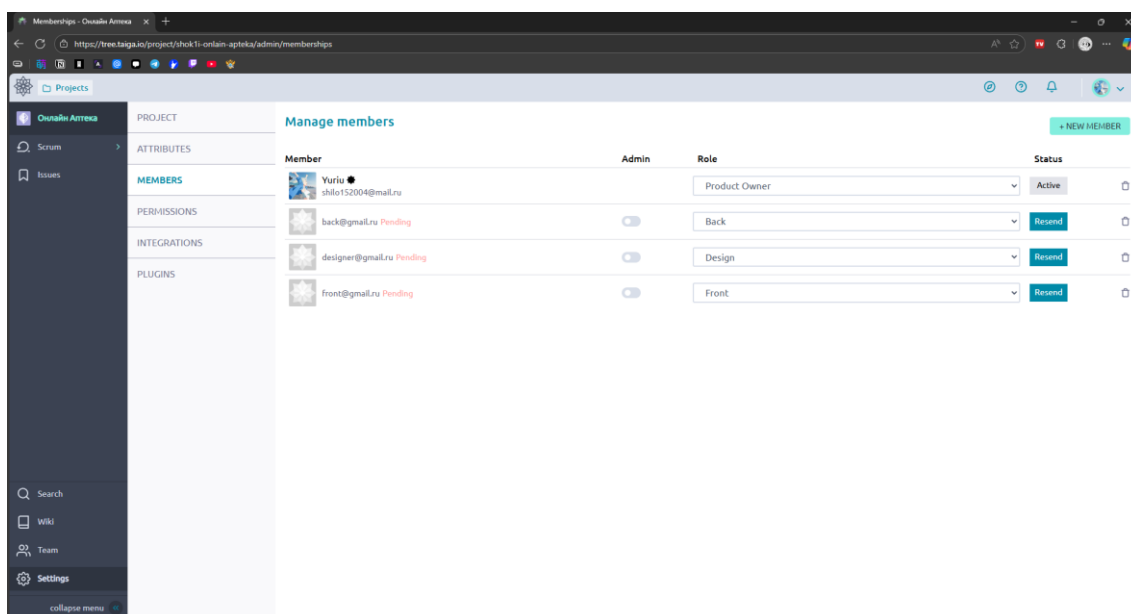


Рисунок 3 – Итоговый список участников

Добавим пользовательские истории, для этого перейдем в Скрам-> Backlog-> пользовательская история (Рисунки 4 – 11).

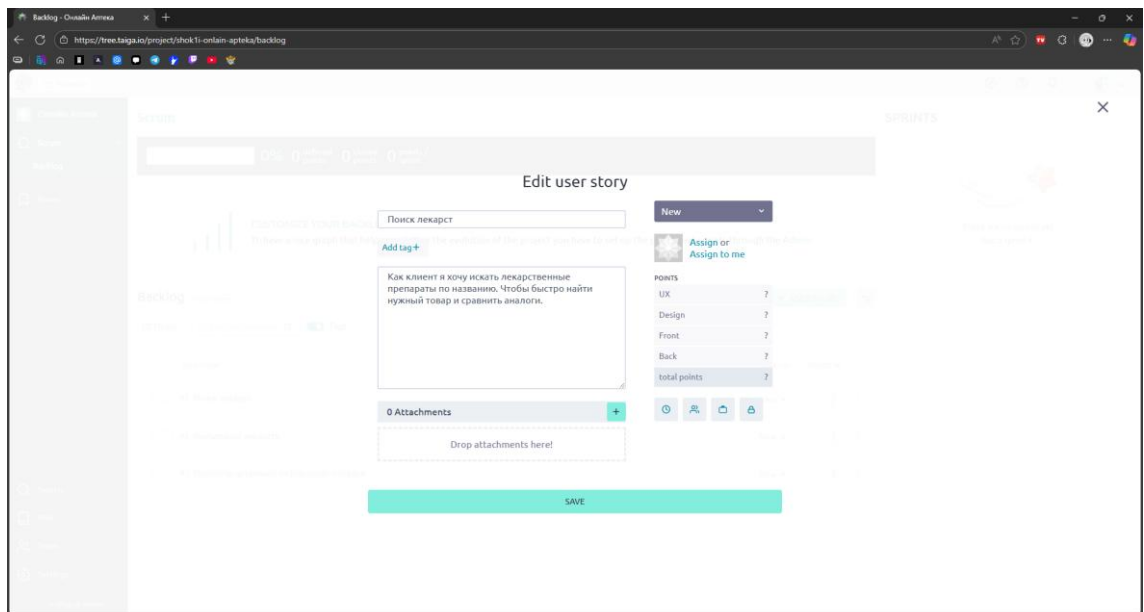


Рисунок 4 – Добавление User Story

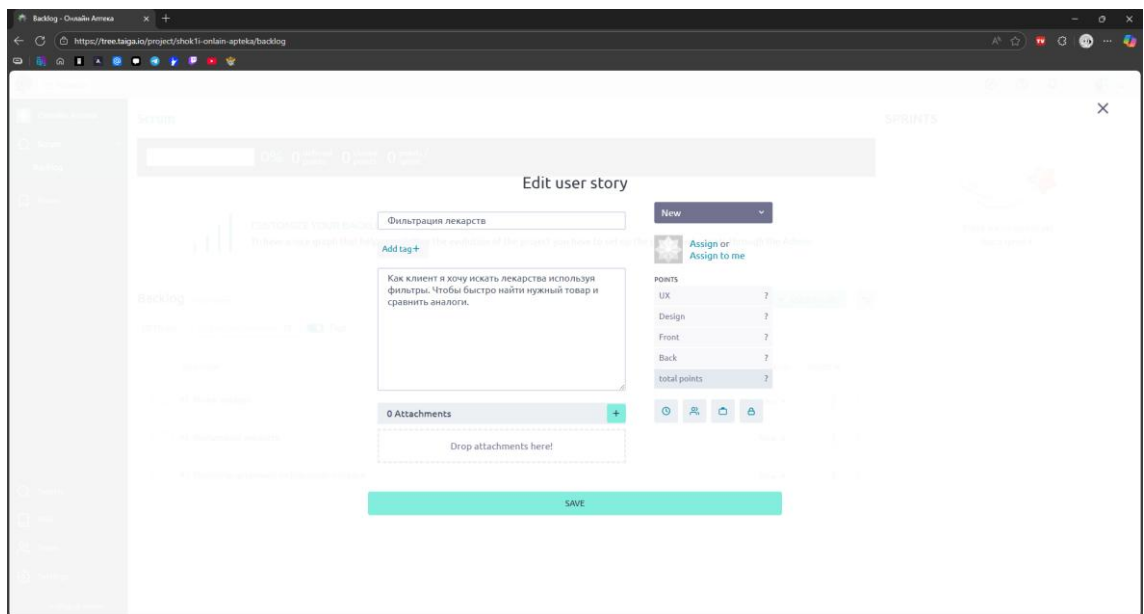


Рисунок 5 – Добавление User Story

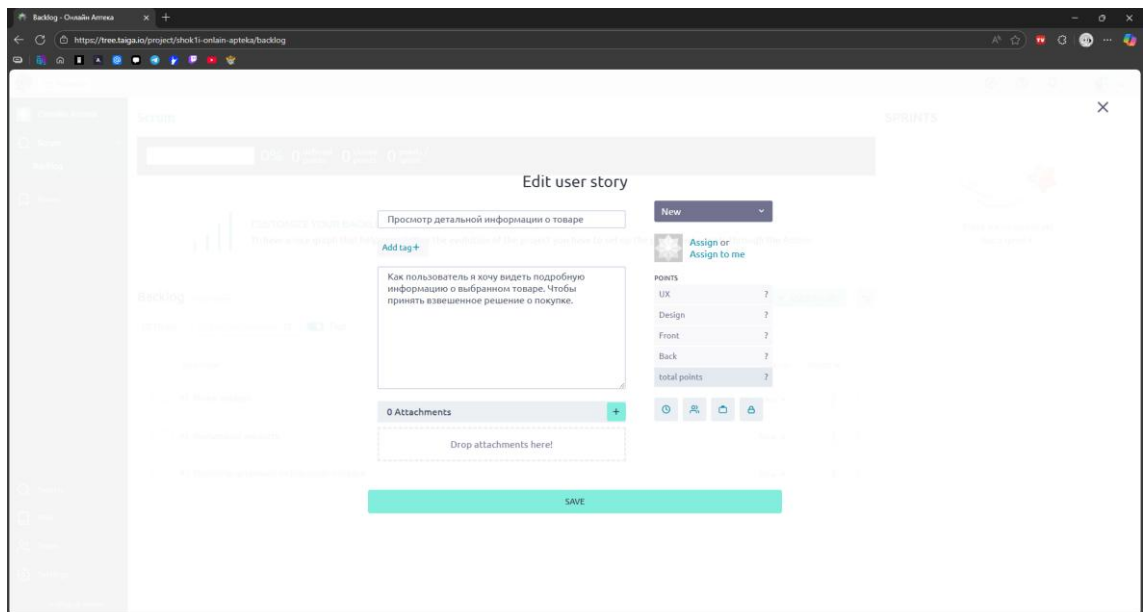


Рисунок 6 – Добавление User Story

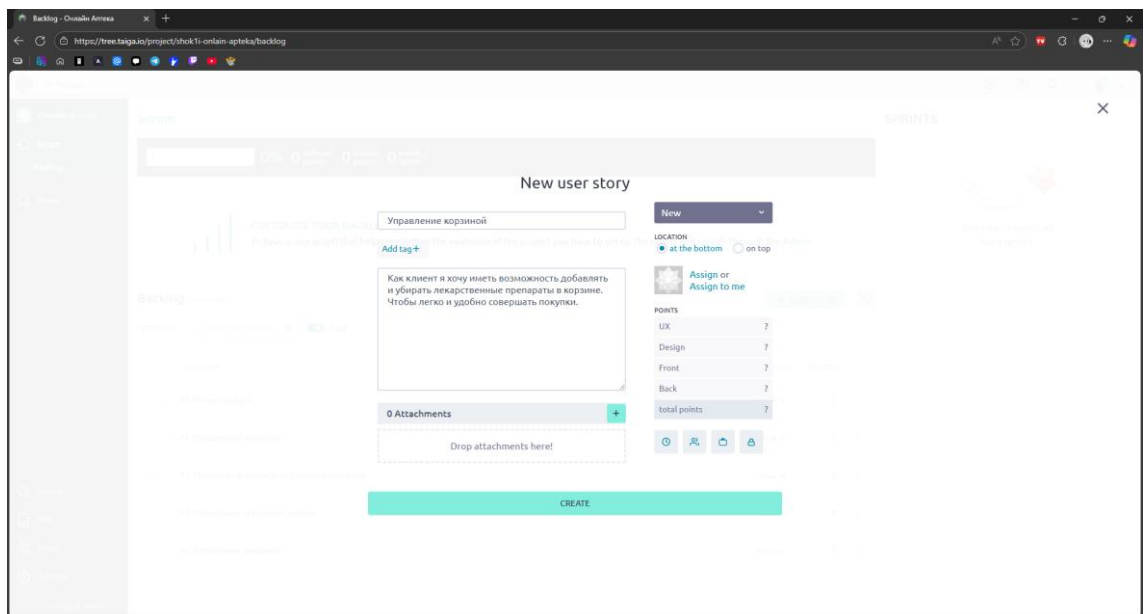


Рисунок 7 – Добавление User Story

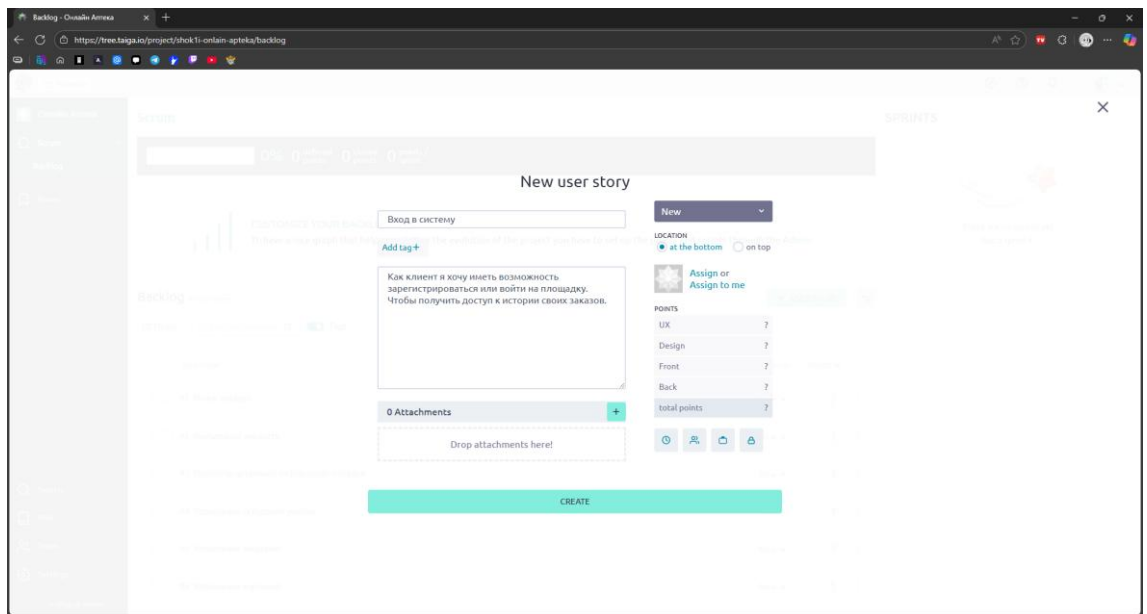


Рисунок 8 – Добавление User Story

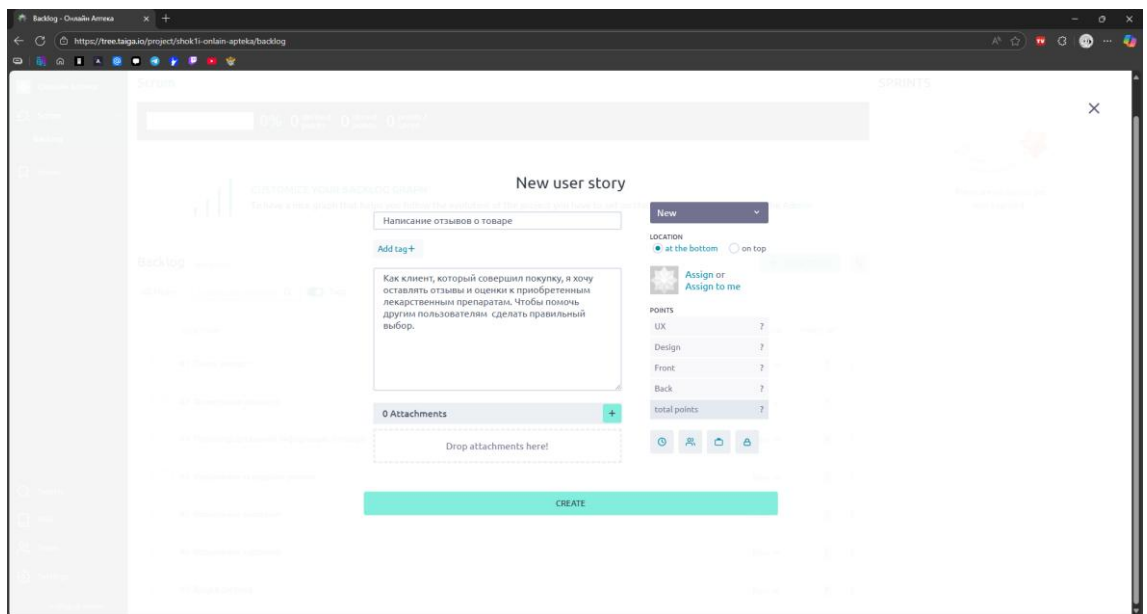


Рисунок 9 – Добавление User Story

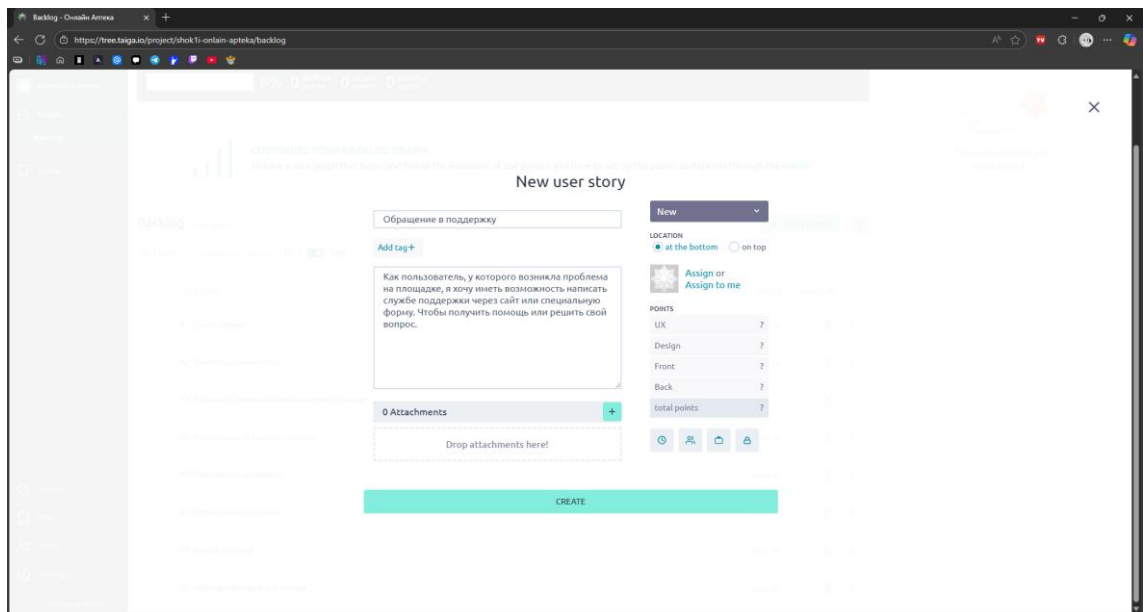


Рисунок 10 – Добавление User Story

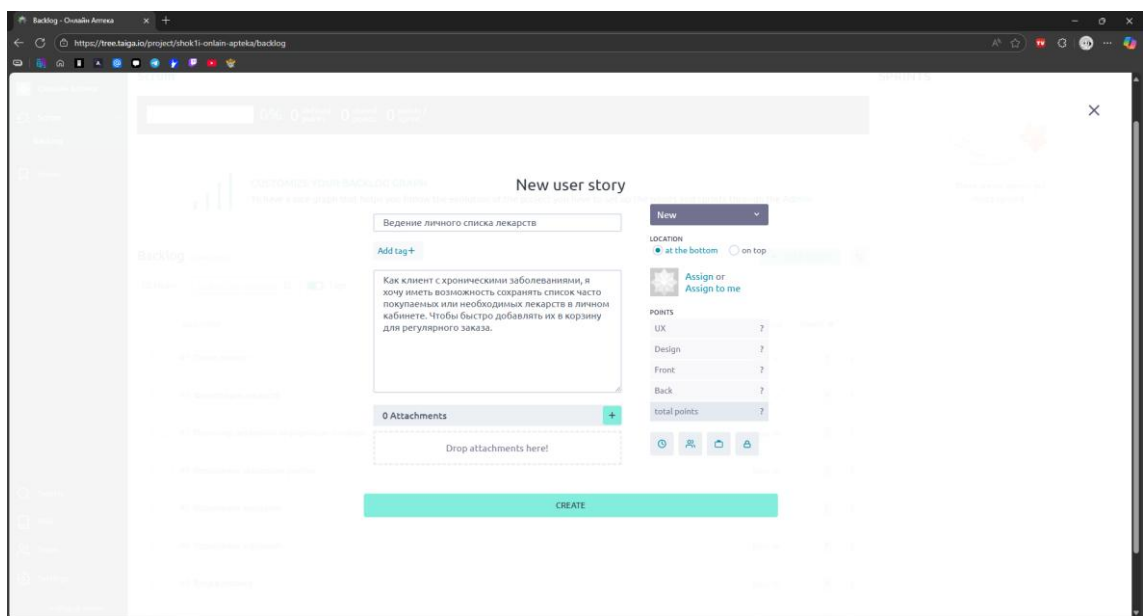


Рисунок 11 – Добавление User Story

Кроме того, добавим тег MVP на особо важные user story, которые являются обязательными для работы проекта. Кроме того, выдадим оценку сложности задачи. Итого, список user story с тегами будет выглядеть так (Рисунок 12).

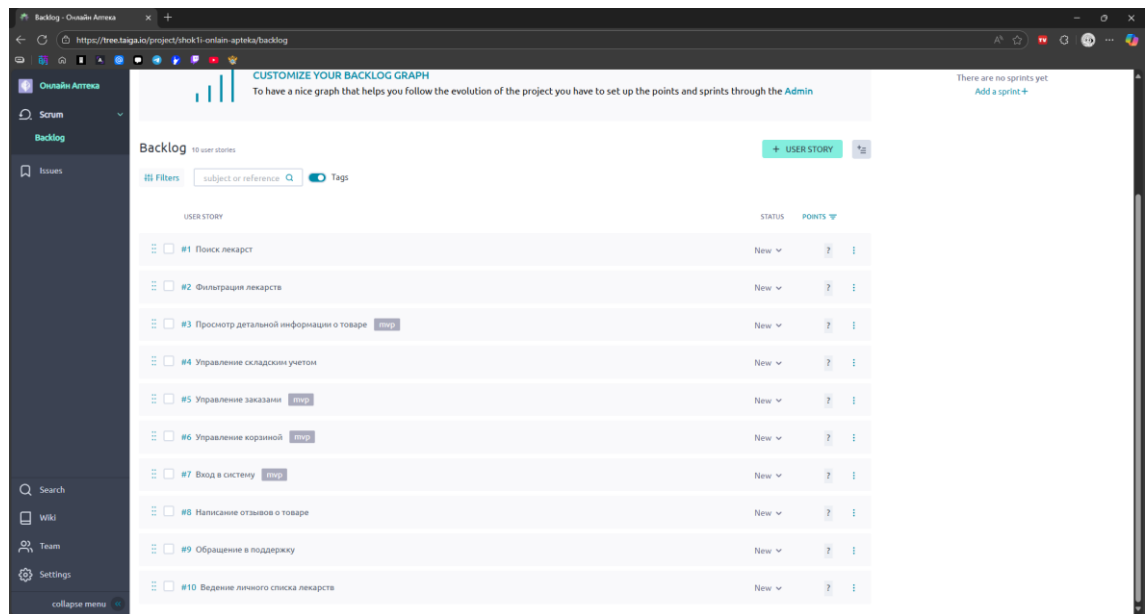


Рисунок 12 – Список User Story

Создадим первый спринт, включим в него основные user story со стороны игрока, выдадим на это трёхнедельный срок (Рисунок 13).

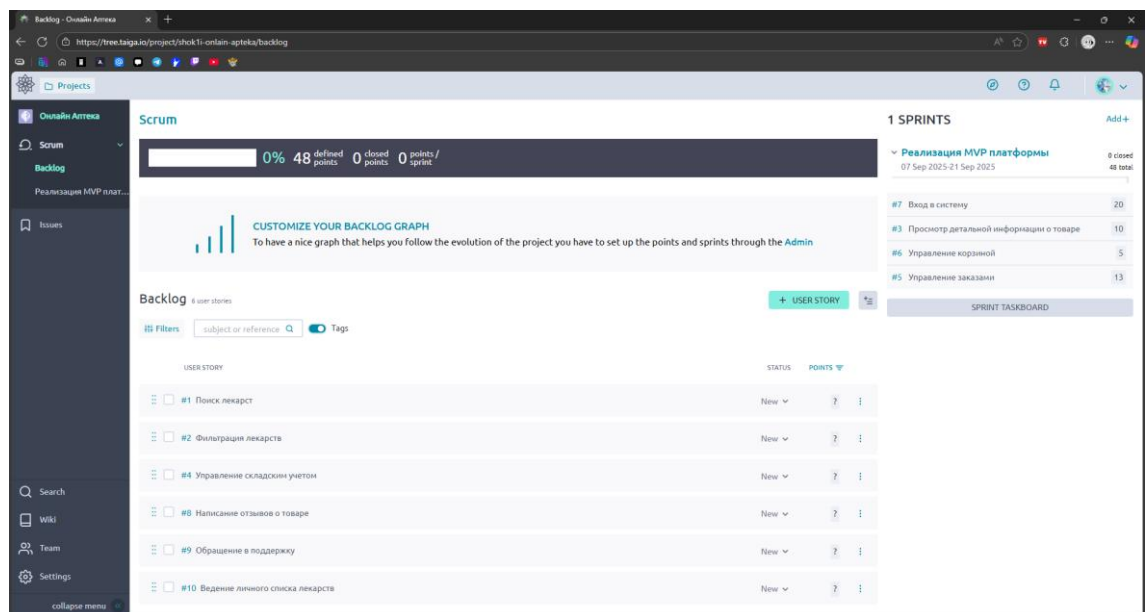


Рисунок 13 – Первый спринт

Добавим новую Task для задач. Так как добавленные пользователи были аккаунтами-пустышками, назначить задачу по факту можно лишь себе самому (Рисунок 14).

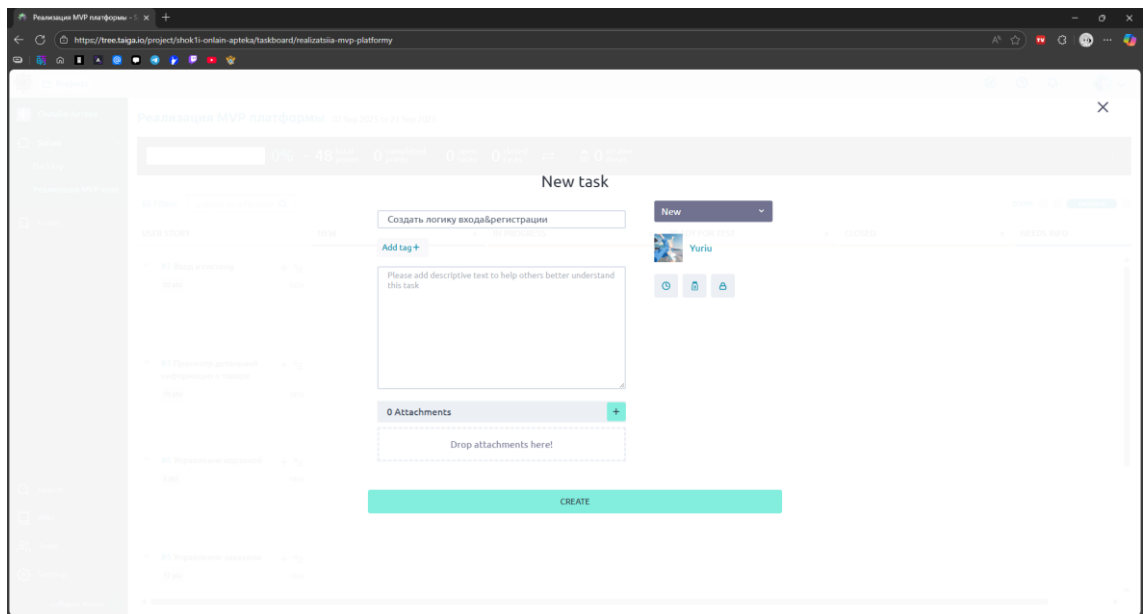


Рисунок 14 – Создание новой задачи

Итого, спринт будет выглядеть так (Рисунок 15).

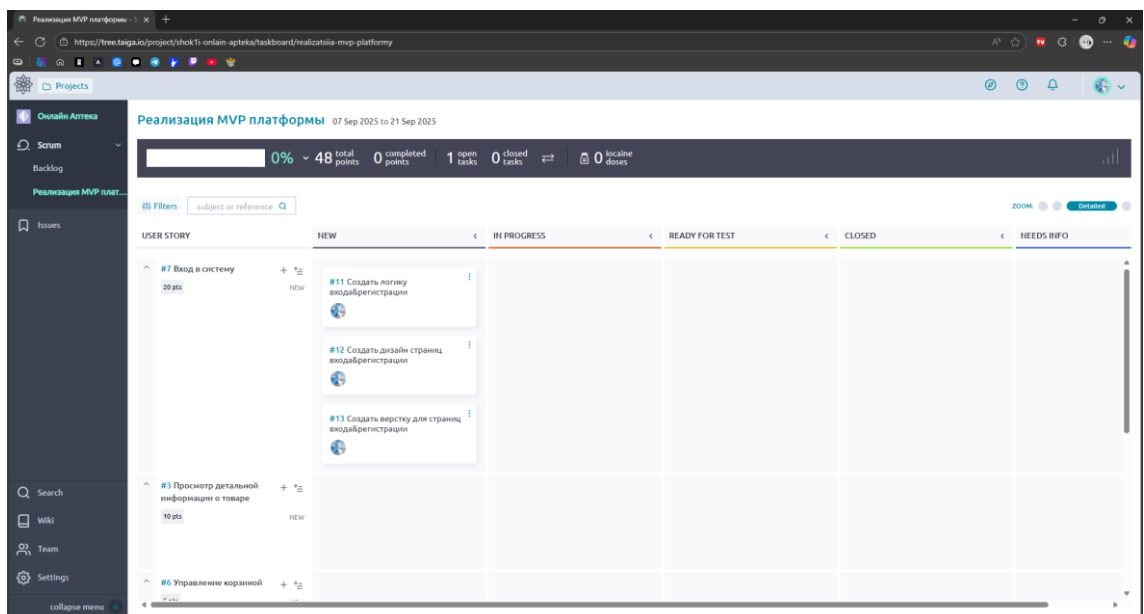


Рисунок 15 – Итоговый первый спринт с задачами

Смоделируем завершение спринта (Рисунок 16).

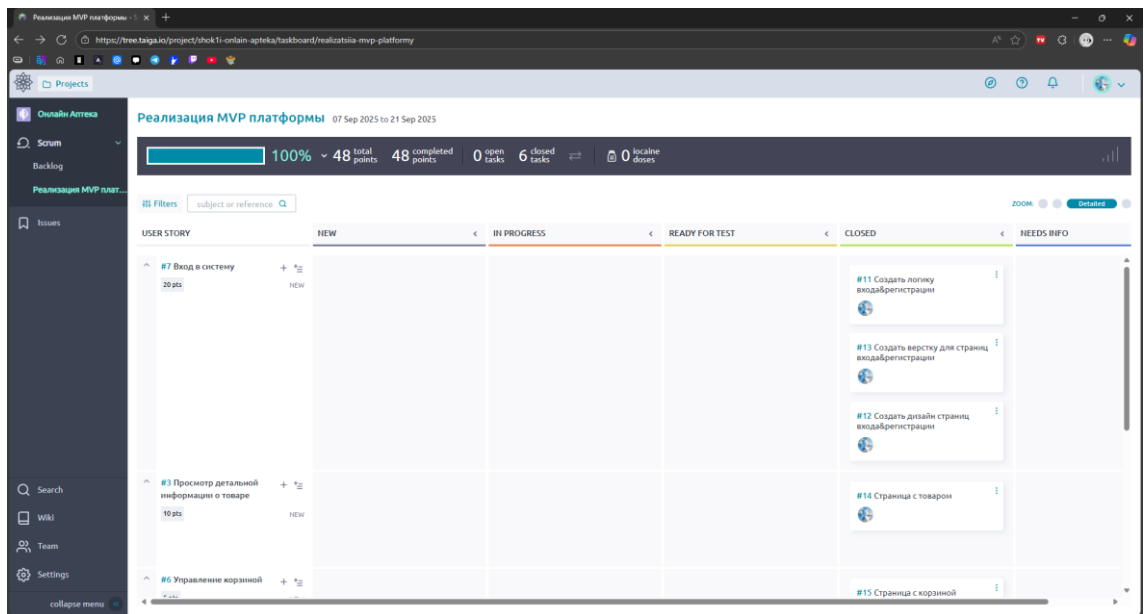


Рисунок 16 – Моделирование завершения спринта

Имитируем совещание для обсуждения прогресса и проблем команды, создадим второй спринт (Рисунок 17).

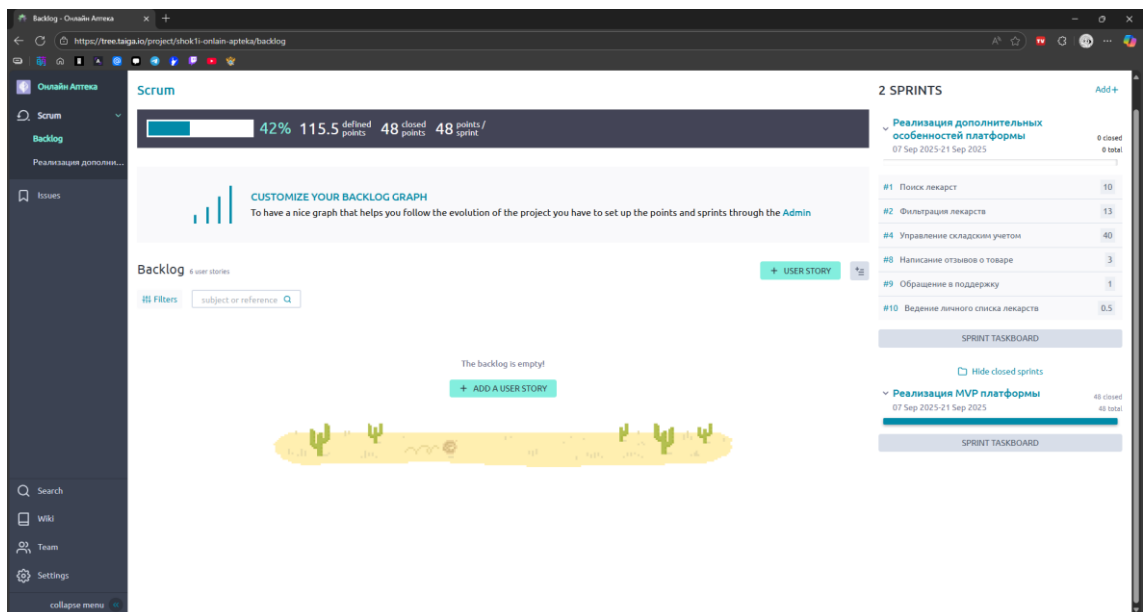


Рисунок 17 – Создание второго спринта

Предположим, что существует проблема, выявленная в ходе обсуждения. Добавим ее в Issues (Рисунок 18).

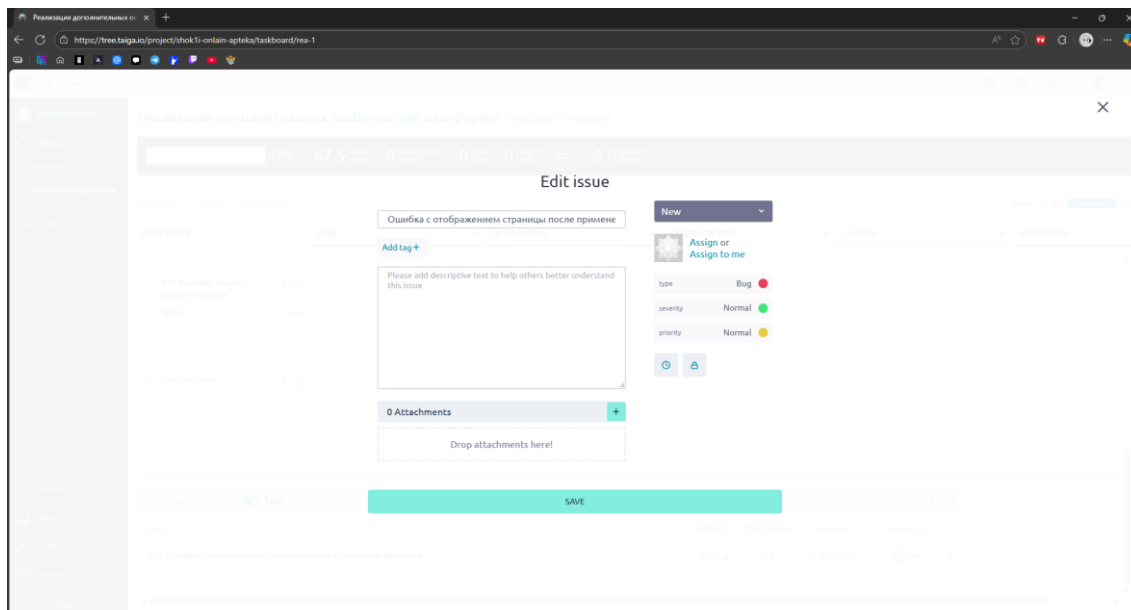


Рисунок 18 – Создание нового issue об ошибке

Добавим задачу, не привязанную к issue, но не имеющую прямого отношения к user story (Рисунок 19).

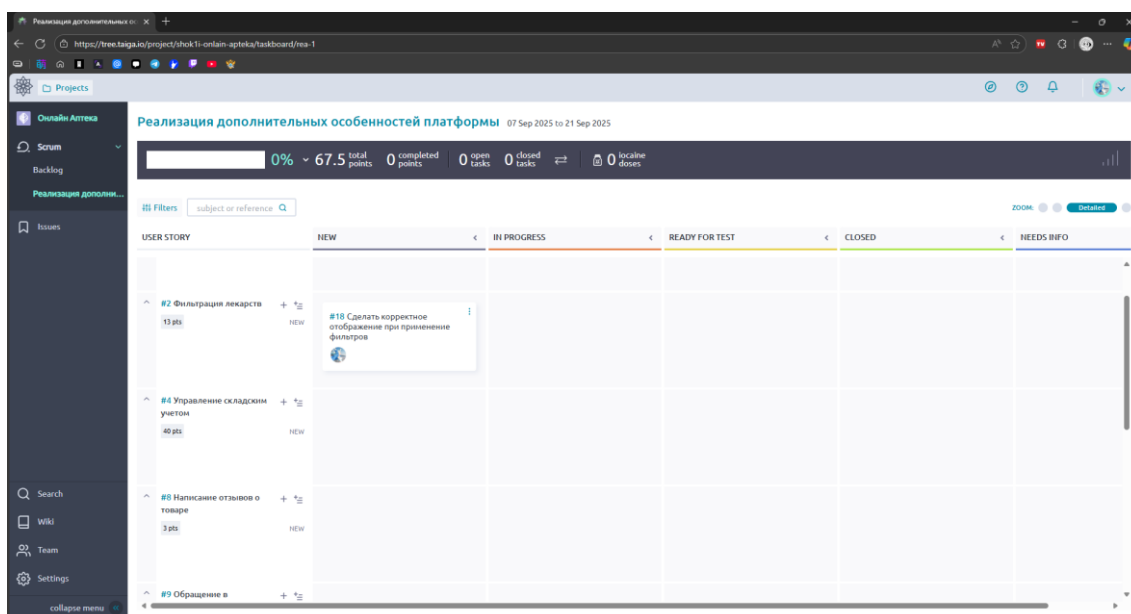


Рисунок 19 – Добавление задачи для решения issue

Добавим задачу для второй user story, перенесем ее в «в процессе» для имитации прогресса (Рисунок 20).

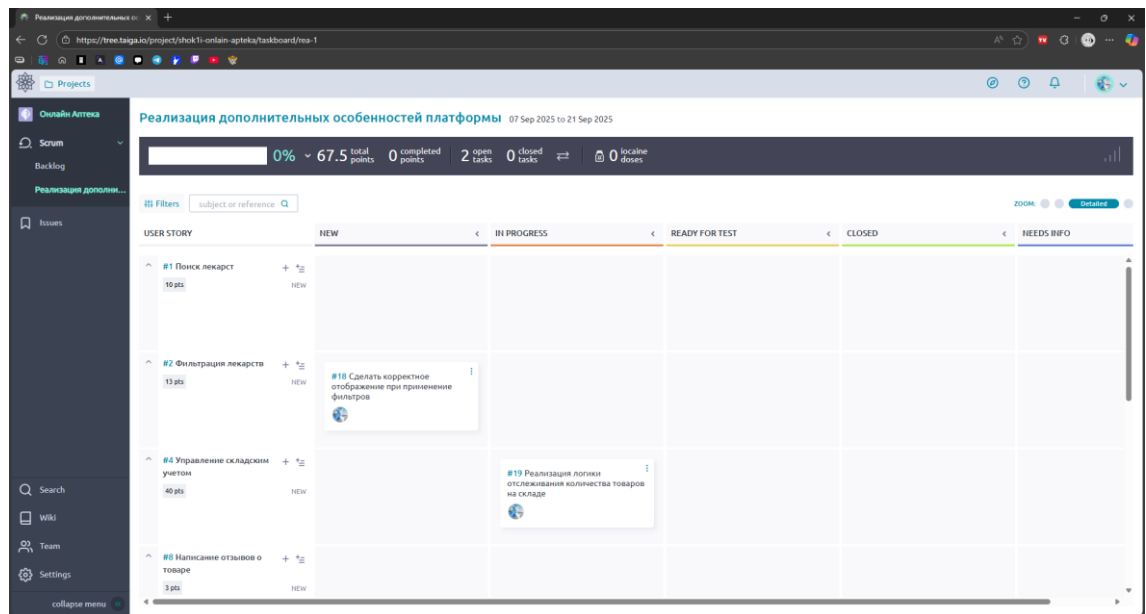


Рисунок 20 – Итоговый второй спринт

Вывод

В ходе выполнения данной практической работы был создан Scrum-проект, созданы user story, задачи для них, сформированные в спринты. Для них были выставлены сроки.

ПРАКТИЧЕСКАЯ РАБОТА 2

Задачи:

Используя данные выше теоретические материалы, вам необходимо организовать и управлять проектом по разработке сложного современного программного обеспечения с использованием методологии Kanban. Конкретный проект и его особенности выбираются из предложенного варианта.

Вам будет необходимо создать новую доску проекта в выбранной системе управления задачами в Taiga.io. По согласованию с преподавателем можно использовать и другие решения, например, Trello. Дайте доске название проекта и определите основные колонки (столбцы), такие как «Бэклог», «В процессе», «Тестирование», «Завершено». Далее, исходя из выбранного проекта, создайте несколько задач, которые необходимо выполнить для разработки приложения. Каждая задача должна содержать описание, приоритет и оценку сложности (если возможно). Распределите задачи по колонкам в соответствии с их текущим статусом. Продемонстрируйте перемещение задачи по доске Канбан в соответствии с их текущим состоянием работы. Добавляйте новые задачи в колонку «Бэклог», перемещайте задачи в колонку «В процессе», когда они начинают выполняться, и так далее. Обновляйте статус задач и следите за их прогрессом. Подробно опишите данные действия. Пригласите других участников проекта в систему и пригласите их к сотрудничеству над задачами, используя встроенные механизмы платформы. Назначайте ответственных за выполнение задач и отслеживайте их работу. Подробно опишите данный процесс. Можно пригласить к проекту однокурсников. После некоторого времени работы с доской Канбан проведите анализ эффективности процесса управления задачами. Попробуйте выявить узкие места и возможности для улучшения процесса, используя для этих целей какой-то метод, например, сбор метрик производительности, такие как время выполнения задач, скорость выполнения

спринтов, количество задач в работе и т. д. Внесите необходимые изменения в структуру доски Канбан и/или процесс работы над задачами.

В результате выполнения данной работы необходимо предоставить преподавателю подробный отчёт, с описанием выполненных шагов.

Результат работы:

Создадим Kanban проект для веб-платформы “Онлайн Аптека”.
(Рисунок 1).

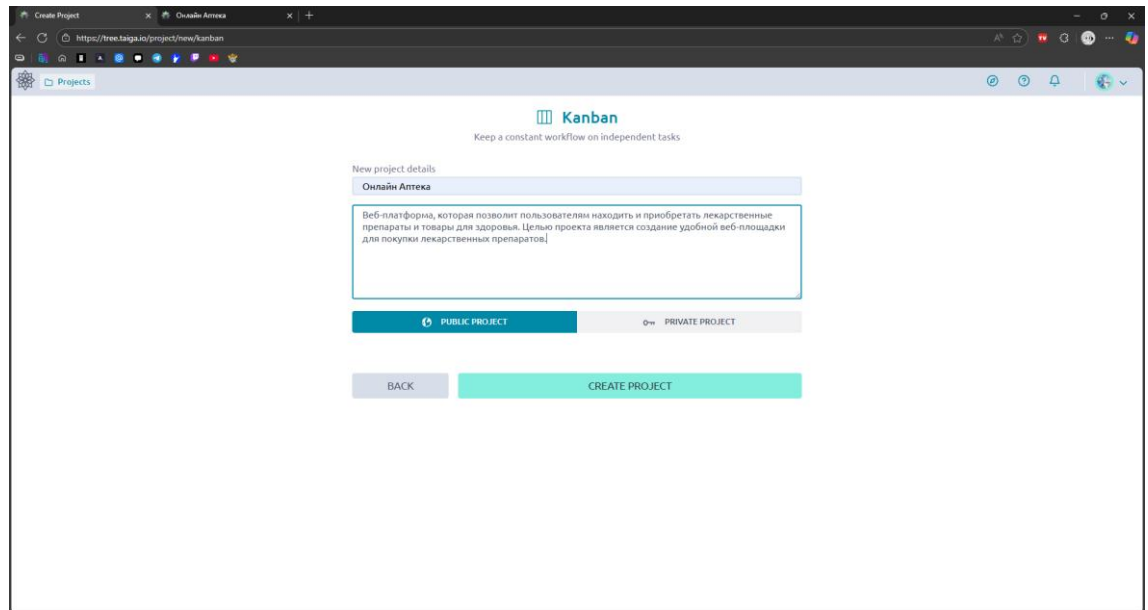


Рисунок 1 – Создание Scrum-проекта

Добавим дополнительных участников в проект. Для этого перейдем в Настройки-> Участники -> новый участник. Также необходимо выбрать роль нового участника (Рисунок 2).

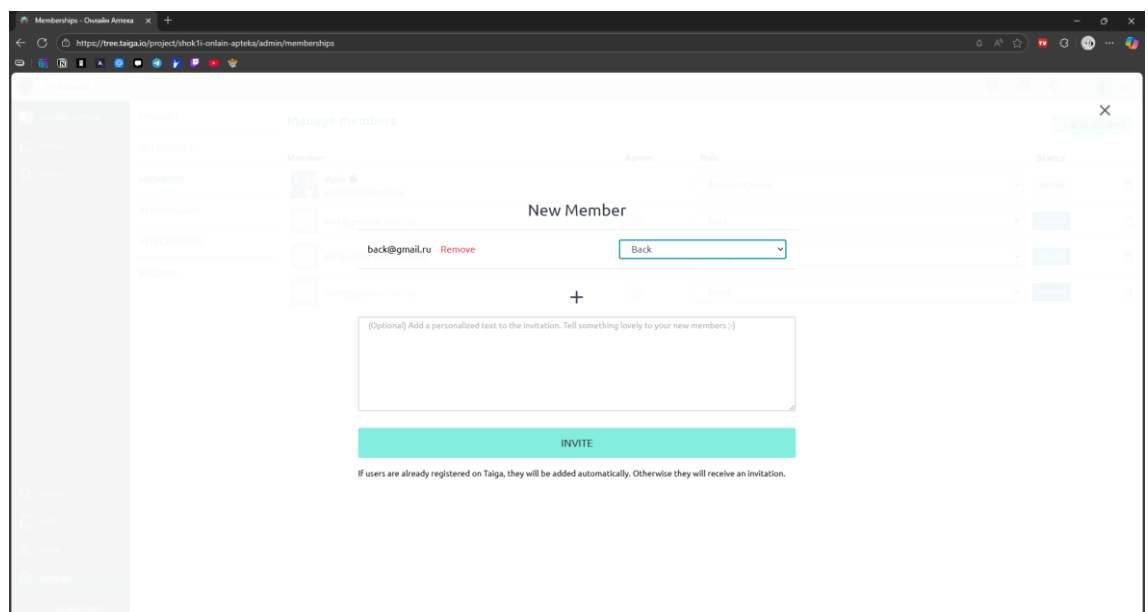


Рисунок 2 – Добавление нового участника

По аналогии добавим разработчика игровой логики, итого получим (Рисунок 3).

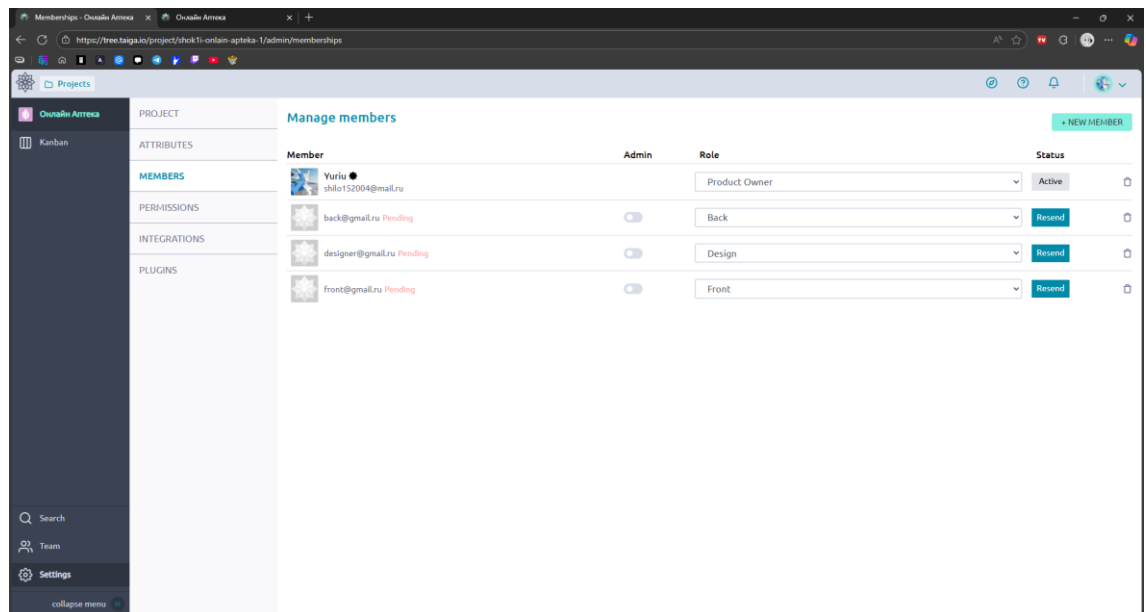


Рисунок 3 – Итоговый список участников

Согласно задаче, нужно создать 4 столбца:

1. Backlog
2. In Progress
3. Test
4. Done

Для этого перейдем в settings->attributes->statuses->user story statuses и изменим их на указанные выше (Рисунок 4).

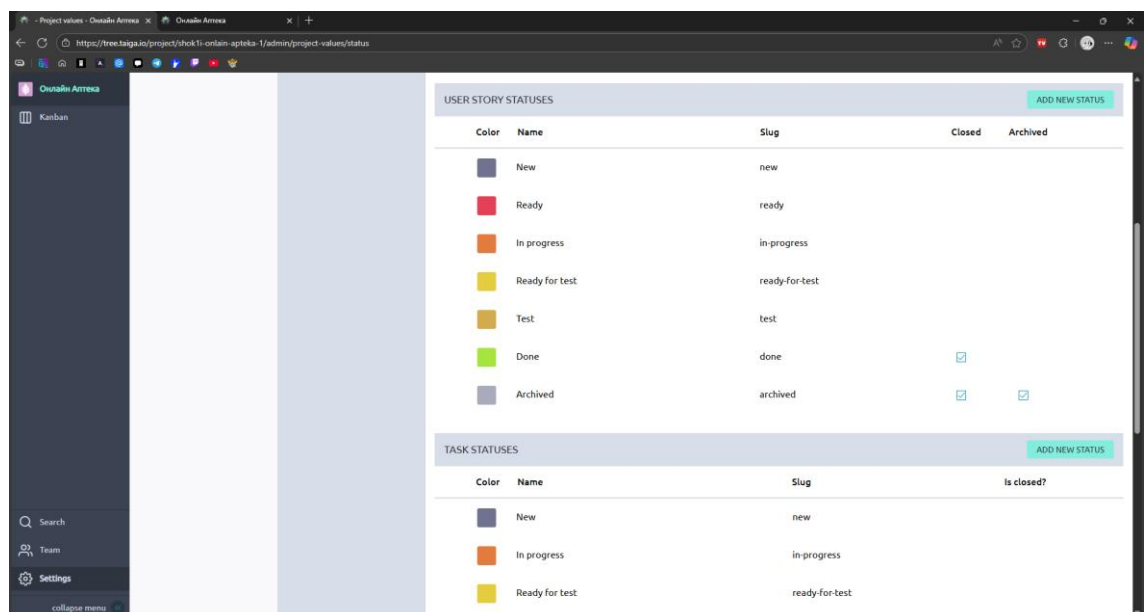


Рисунок 4 – Изменение столбцов

Добавим пользовательские истории, для этого перейдем в Kanban-> Backlog-> пользовательская история (Рисунки 5 – 12).

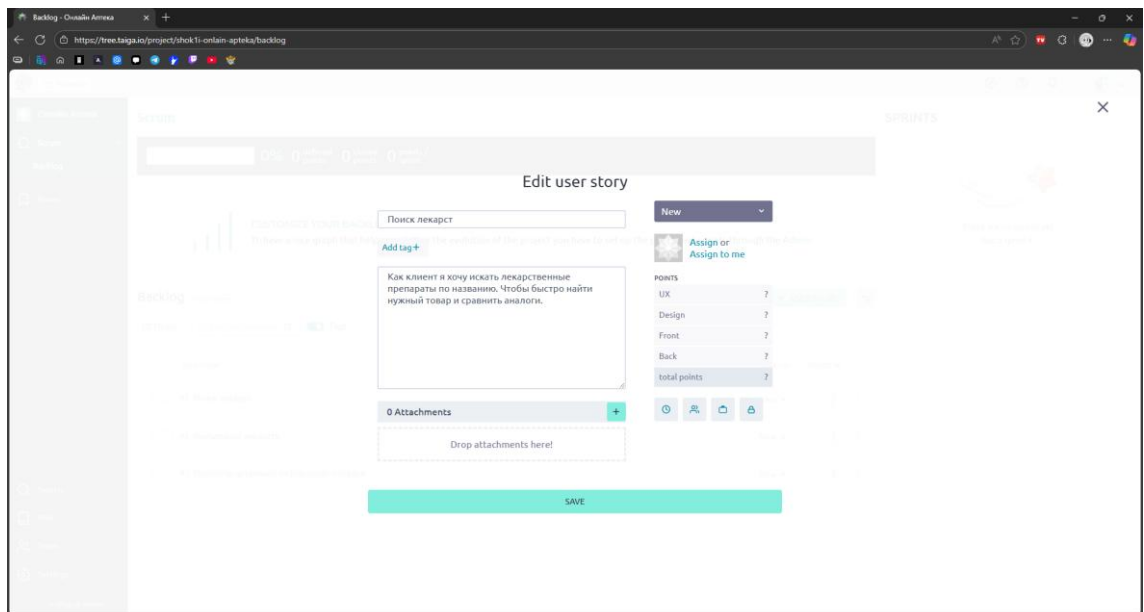


Рисунок 4 – Добавление User Story

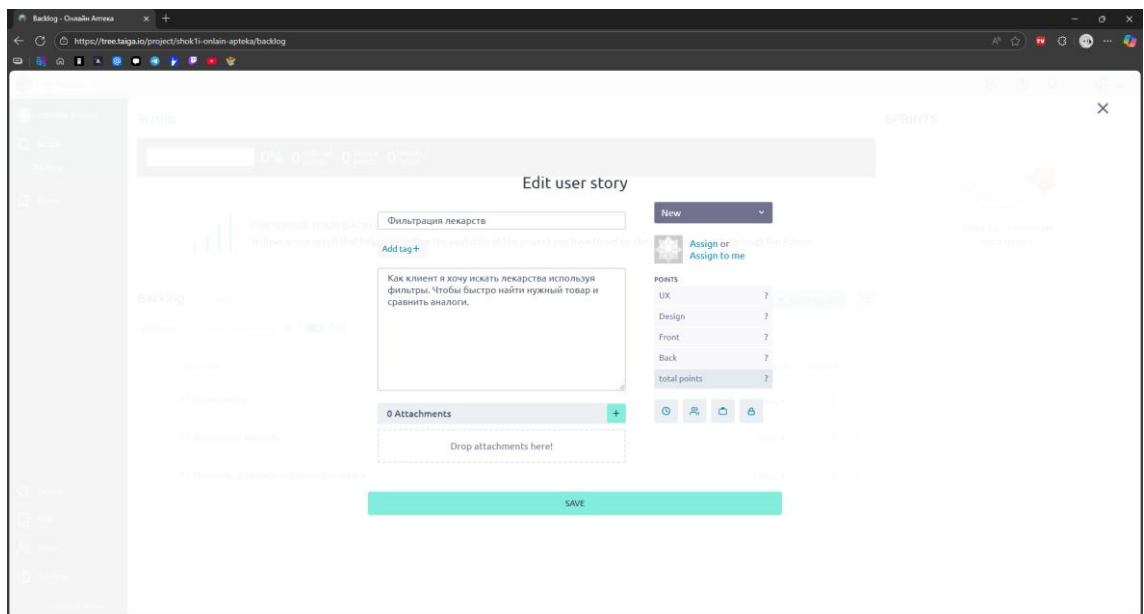


Рисунок 5 – Добавление User Story

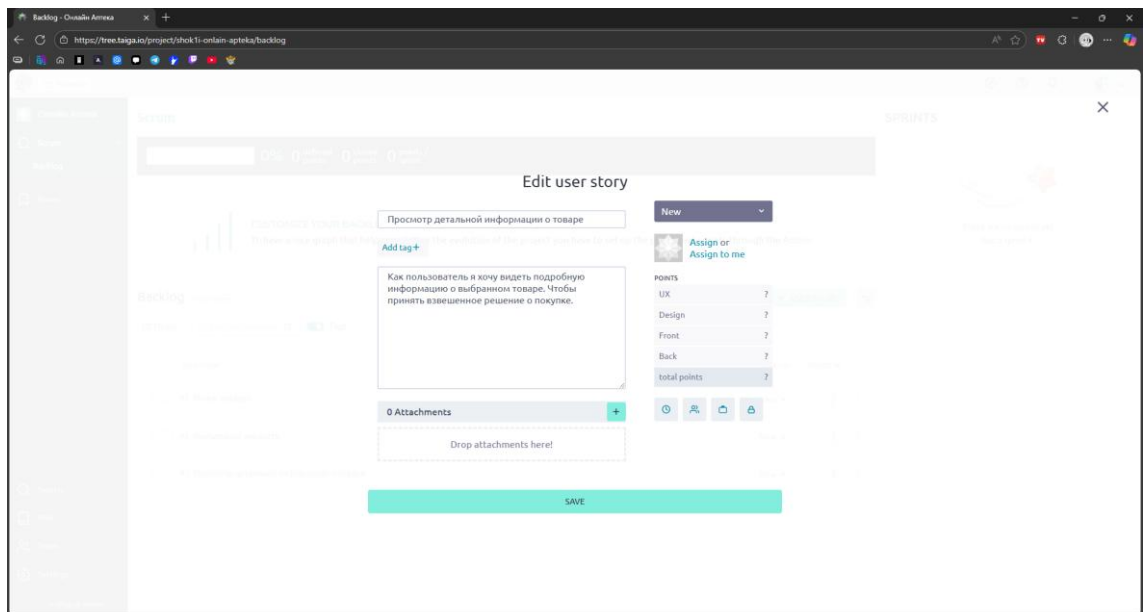


Рисунок 6 – Добавление User Story

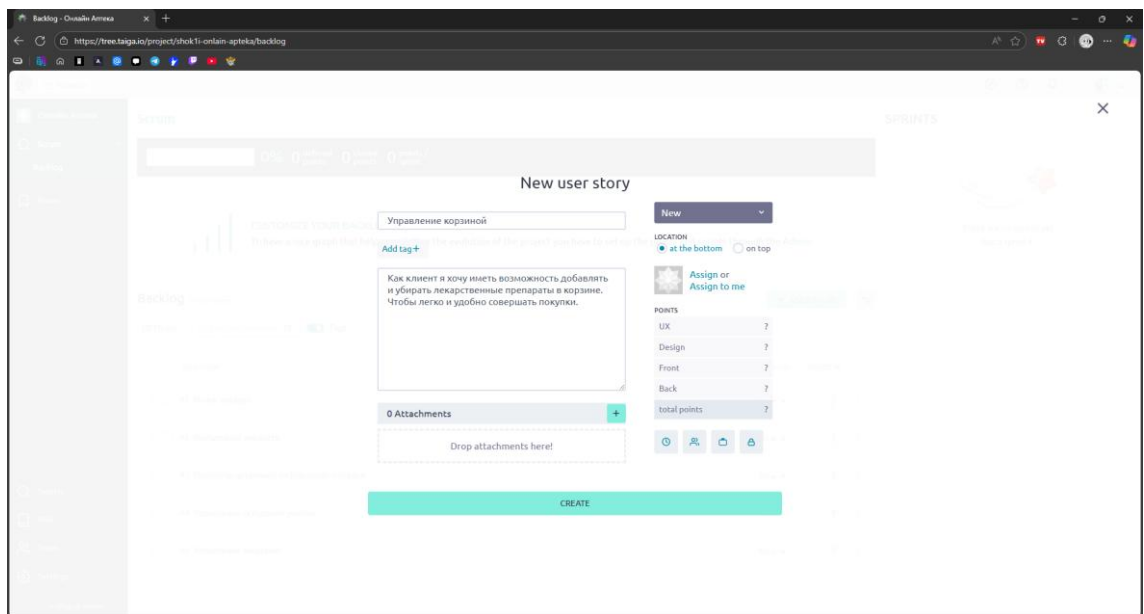


Рисунок 7 – Добавление User Story

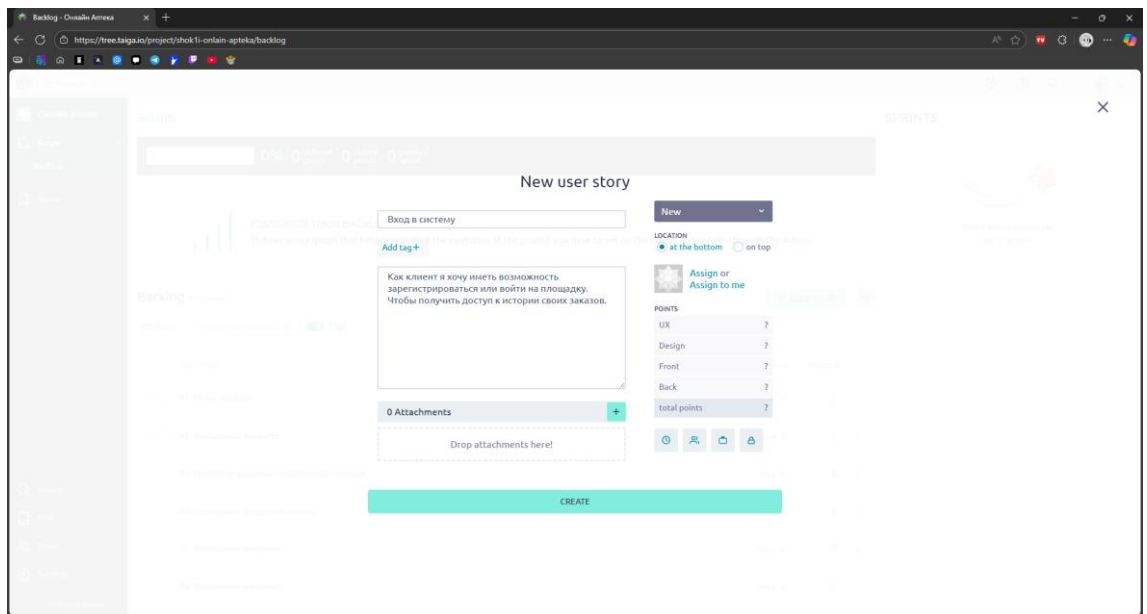


Рисунок 8 – Добавление User Story

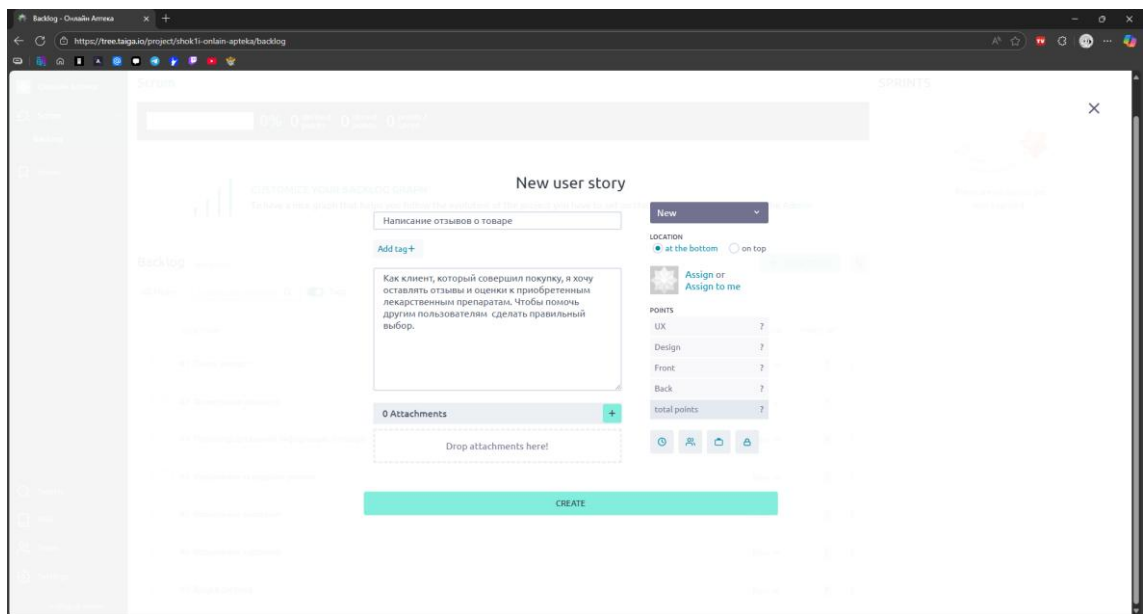


Рисунок 9 – Добавление User Story

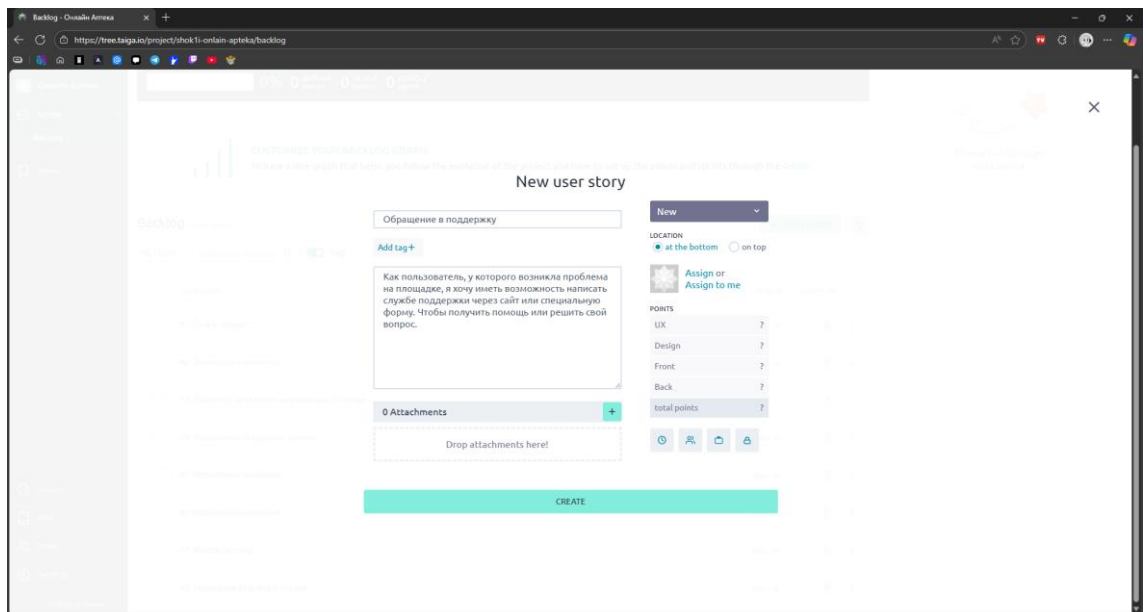


Рисунок 10 – Добавление User Story

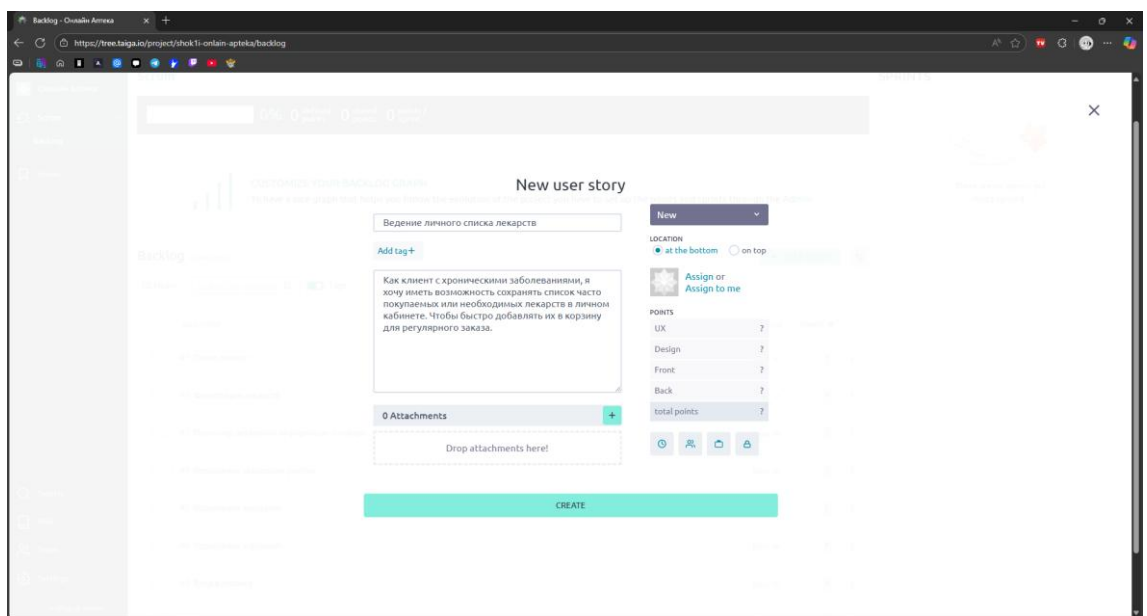


Рисунок 11 – Добавление User Story

Выставим роли на имеющиеся user story. Переместим user story в соответствующие их статусу выполнения колонны (Рисунок 12).

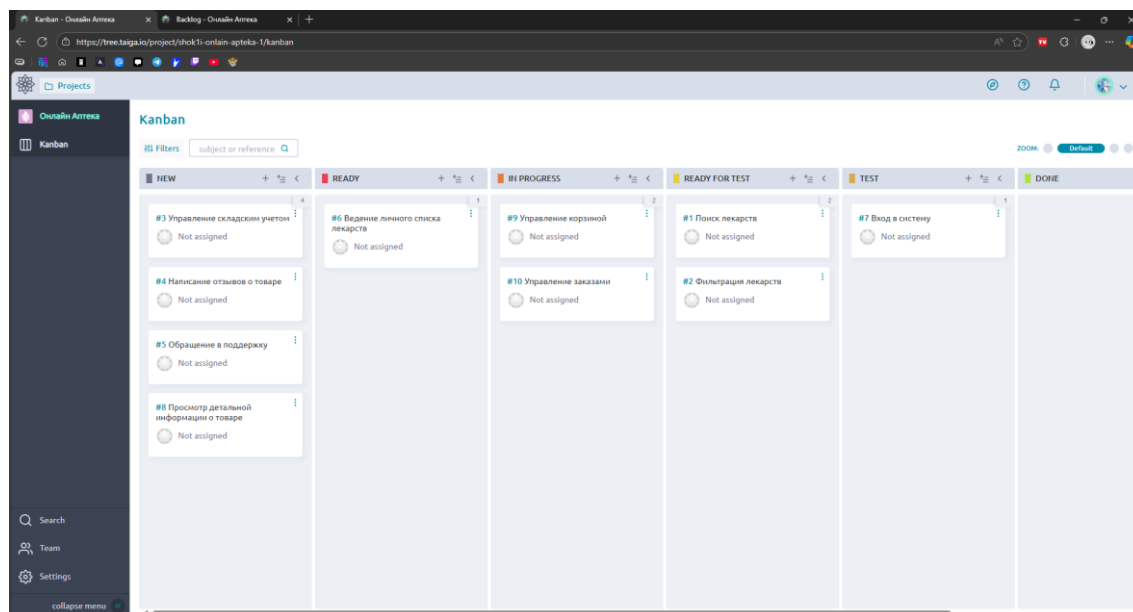


Рисунок 12 – Список User Story

После тестирования входа на веб-платформу, задача была закрыта. Поиск и фильтрация лекарств перемещается на этап тестирования, начинается работа над просмотром детальной информации о проекте. Введение личного списка лекарств готов к тестированию (Рисунок 13).

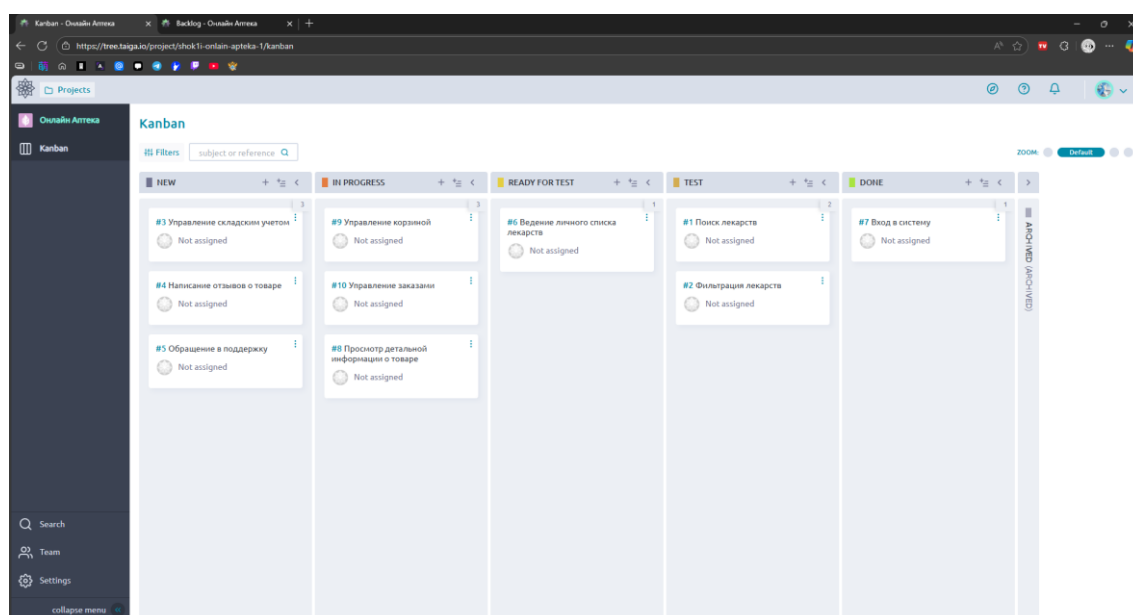


Рисунок 13 – Загрузка материалов в процессе

Поиск и фильтрация лекарств по результатам тестирования возвращается на исправление. Управление корзиной и заказами переходит на этап тестирования (Рисунок 14).

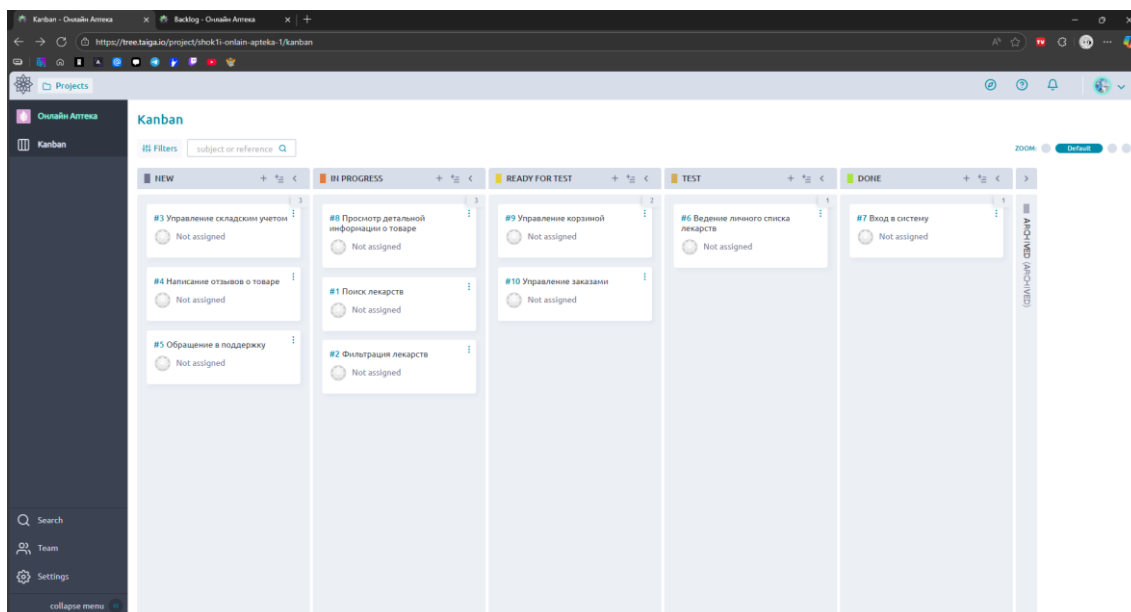


Рисунок 14 – Создание новой задачи

В ходе работы выявлена не очень удобная особенность: при возвращении задачи на доработку, она смешивается с остальными задачами, что не акцентирует внимания на том, что ее нужно переделать. Может возникнуть ситуация, когда работник просто подумает, что забыл отменить карточку как отправленную на тестировку. Для этого предлагается ввести новую колонку Revision и переместить туда задачи поиска и фильтрации товаров (Рисунок 15).

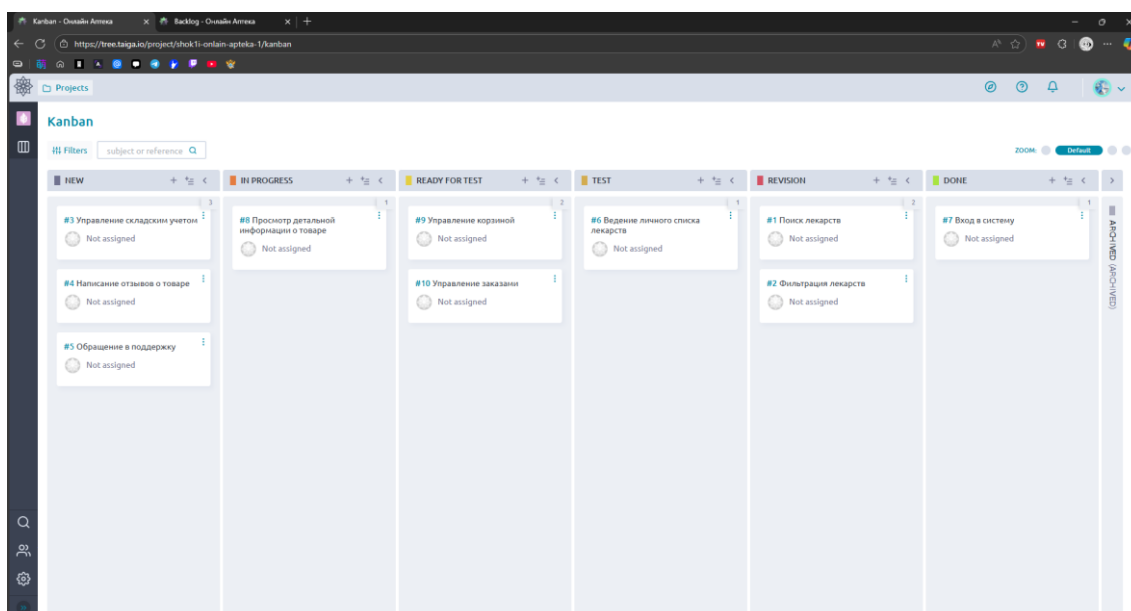


Рисунок 15 – Доска после добавления нового столбца

Вывод

В ходе выполнения данной практической работы был создан Kanban-проект, созданы user story, им были выданы исполнители. Была проанализирована работа с канбан доской, в результате чего был добавлен еще один столбец.

ПРАКТИЧЕСКАЯ РАБОТА 3

Создадим проект в Obsidian для разработки веб-платформы “Онлайн аптека”.

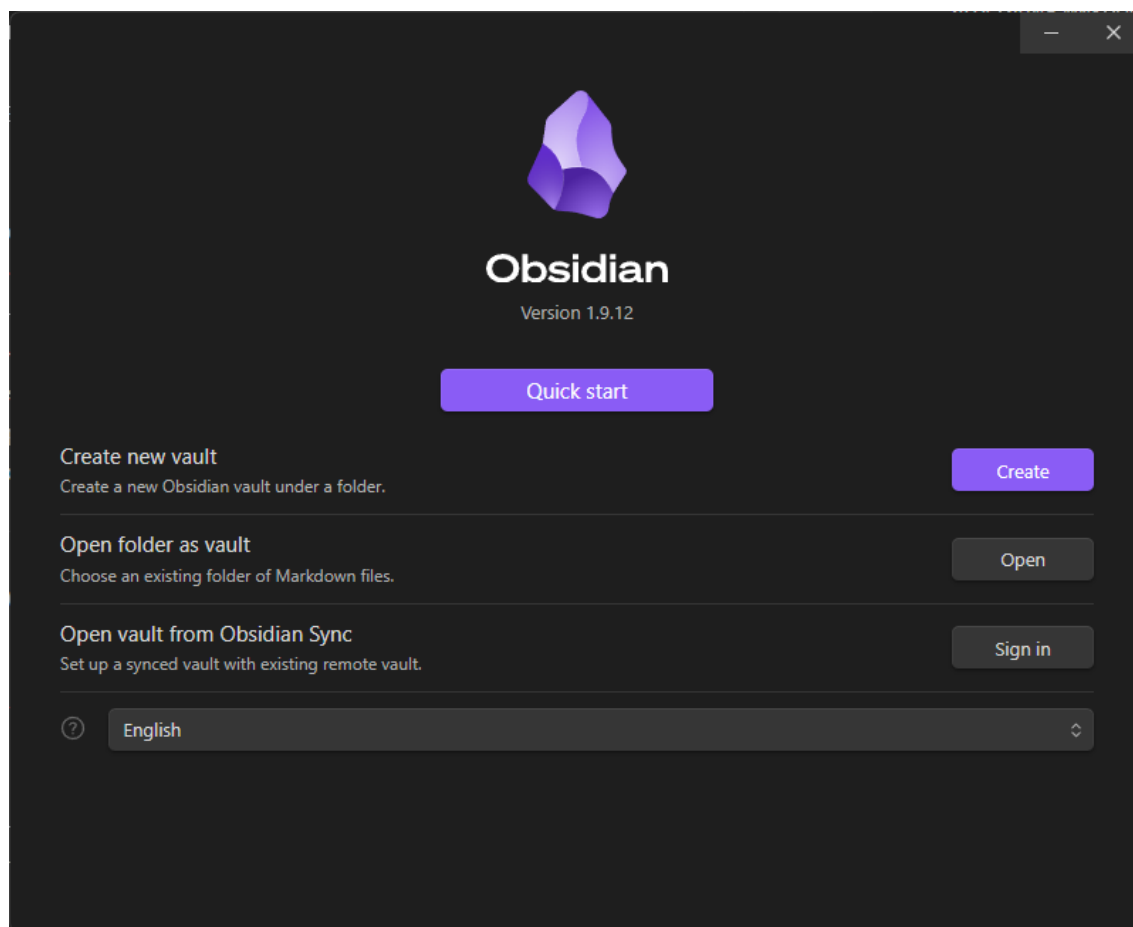


Рисунок 1 – Первоначальная настройка хранилища

Создадим главную страницу нашего хранилища. Здесь у нас будут находиться внешние ссылки и описание нашего хранилища (Рисунок 2)

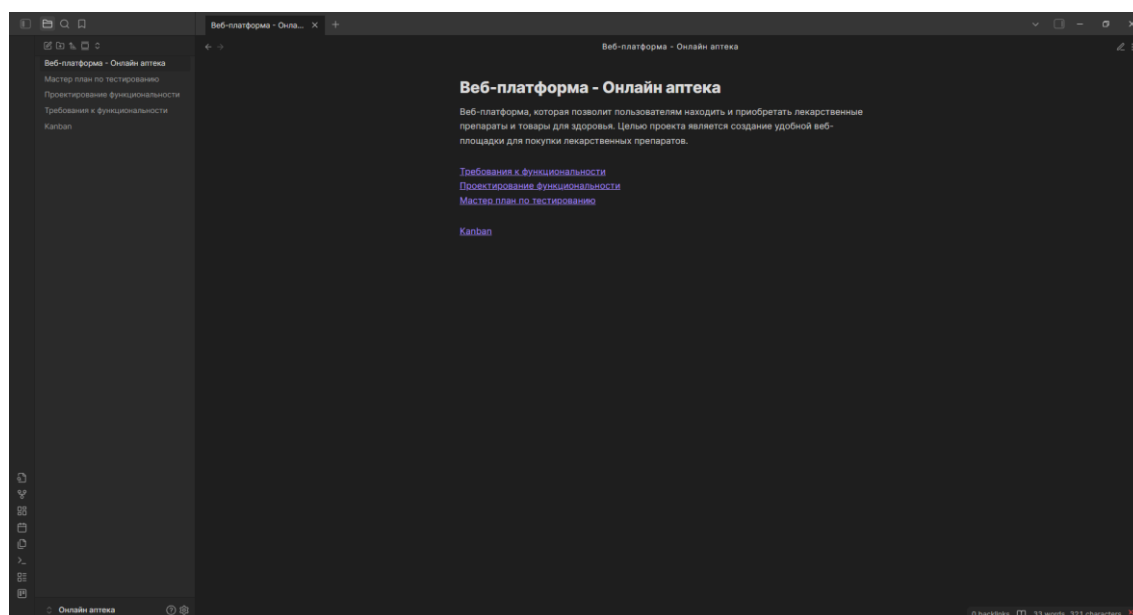


Рисунок 2 – Главный файл проекта

В новой заметке опишем функциональность модуля основного функционала. В нем будут описаны основные функции, особенности интерфейса, требования к производительности и безопасности. Ссылку на заметку добавим на главную страницу (Рисунок 3).

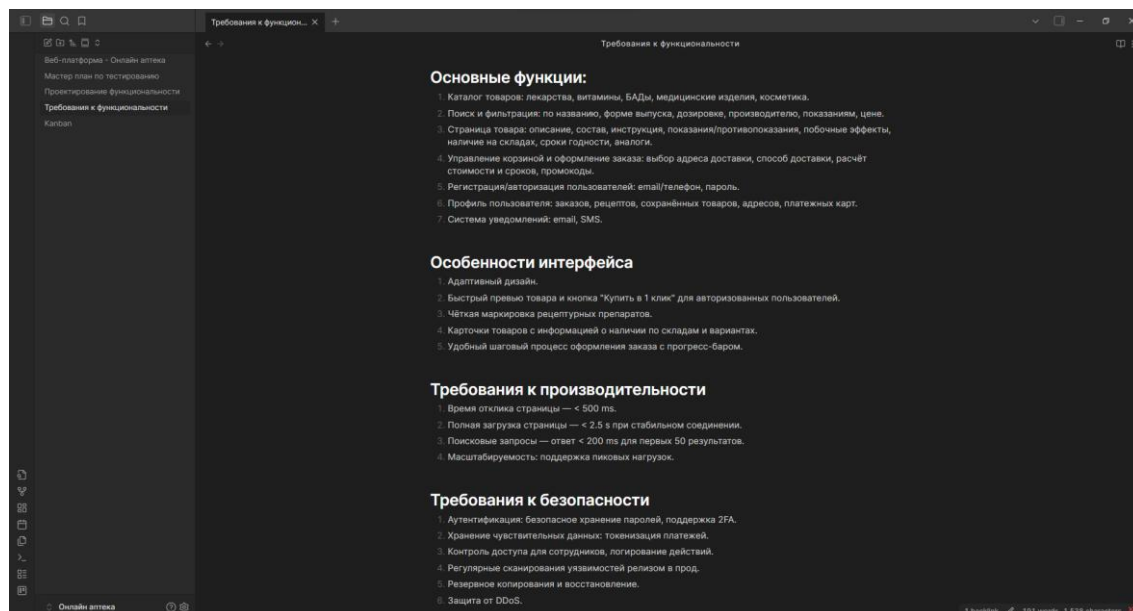


Рисунок 3 – Содержание файла “Требования к функциональности”

Также опишем проектирование функциональности, а именно, информацию о языке, фреймворке и базе данных. Ссылку на заметку добавим на главную страницу (Рисунок 4)

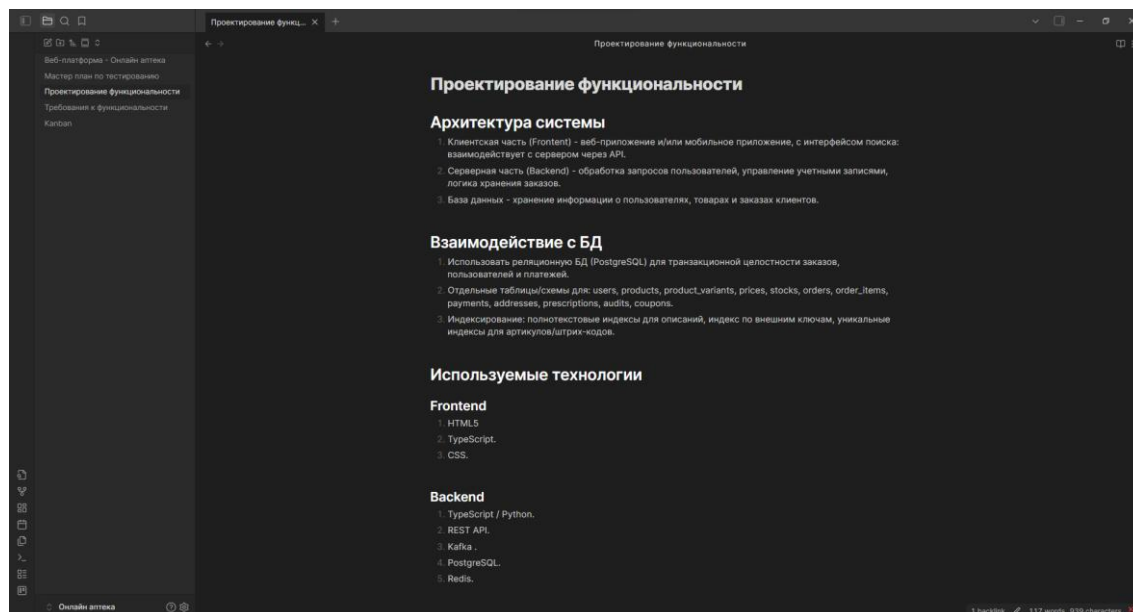


Рисунок 4 – Содержание файла “Проектирование функциональности”

Опишем мастер-план тестирования, добавим ссылку на него в главной странице (Рисунок 5).

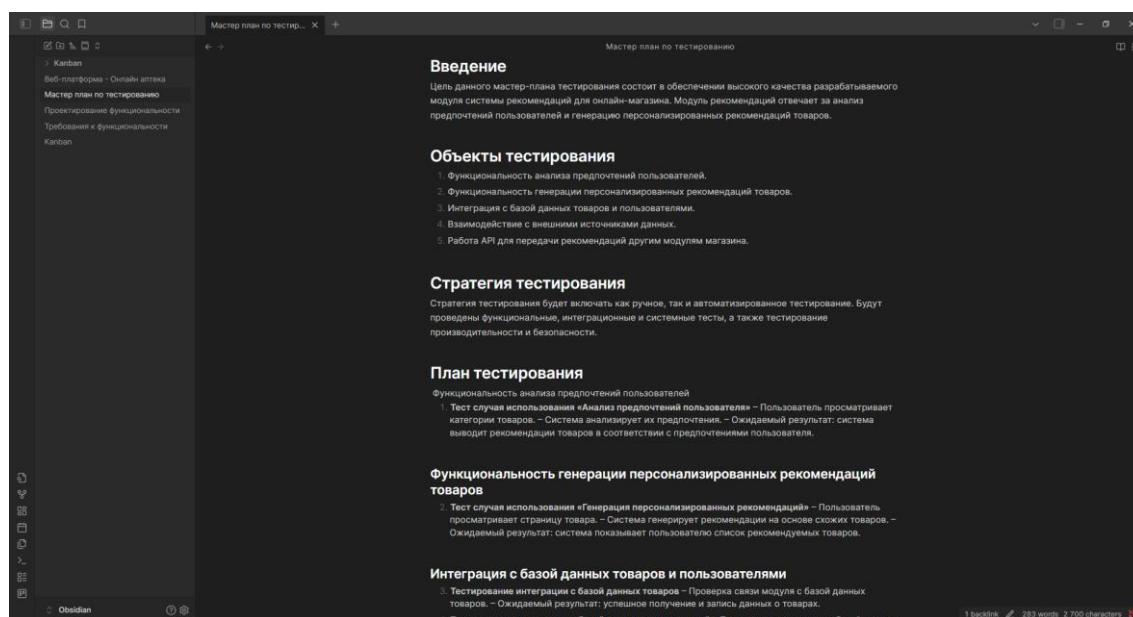


Рисунок 5 – Содержание файла “Мастер план по тестированию”

Установим пользовательское расширение Kanban для создания одноименной доски. После чего добавим задачи, симулируем успешный прогресс при их выполнении. Выдадим им сроки (Рисунок 6).

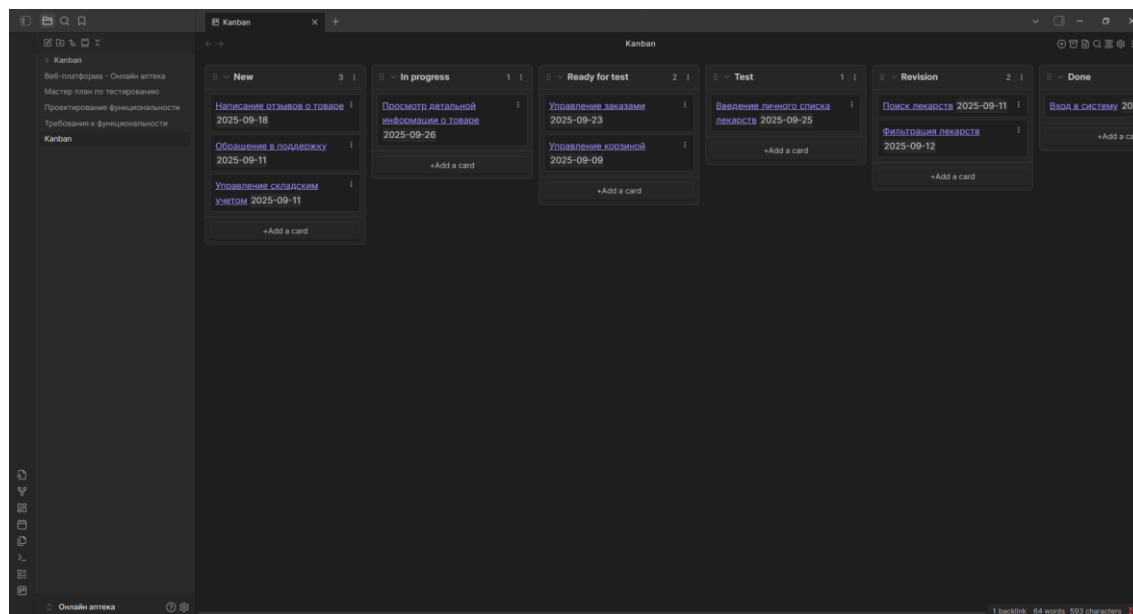


Рисунок 6 – Содержание файла “Kanban”

Опишем каждое задание с помощью собственной заметки. Пример данной заметки показан на рисунке 7.

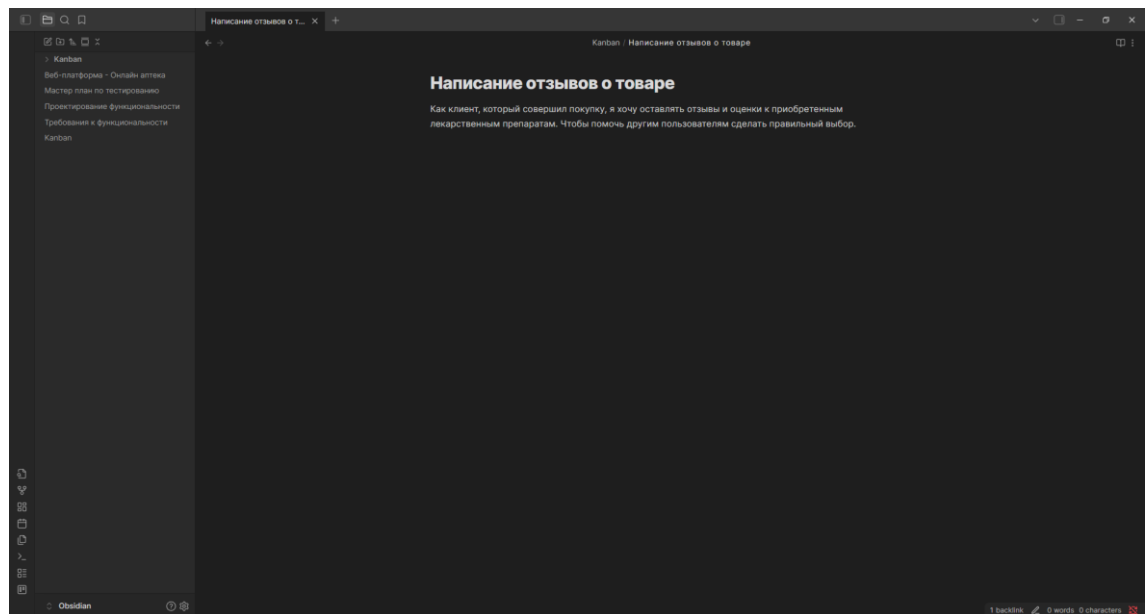


Рисунок 7 – Содержание файла “Написание отзывов о товаре”

На рисунке 8 показан итоговый граф созданный при выполнении практической работы.



Рисунок 8 –Итоговый граф, получившегося хранилища

Вывод

В ходе выполнения данной практической работы был создан Obsidian проект, в котором описаны функциональные требования, мастер-план тестирования, прогресс выполнения и проектирование функциональности. Была использована Kanban-доска для визуализации прогресса выполнения.

ПРАКТИЧЕСКАЯ РАБОТА 4

Задачи:

Используя информацию представленную выше вам необходимо создать Jenkins Pipeline, который будет имитировать сборку проекта из github репозитория, то есть выполнить шаги показанные в методических указаниях. Результатом работы должен быть созданный репозиторий, функционирующий Jenkins Pipeline, описанный с помощью Jenkinsfile, описывающий весь процесс CI/CD.

Результат работы:

Установим докер образ Jenkins с помощью команды “docker pull jenkins/jenkins:lts-alpine-jdk17” (Рисунок 1).

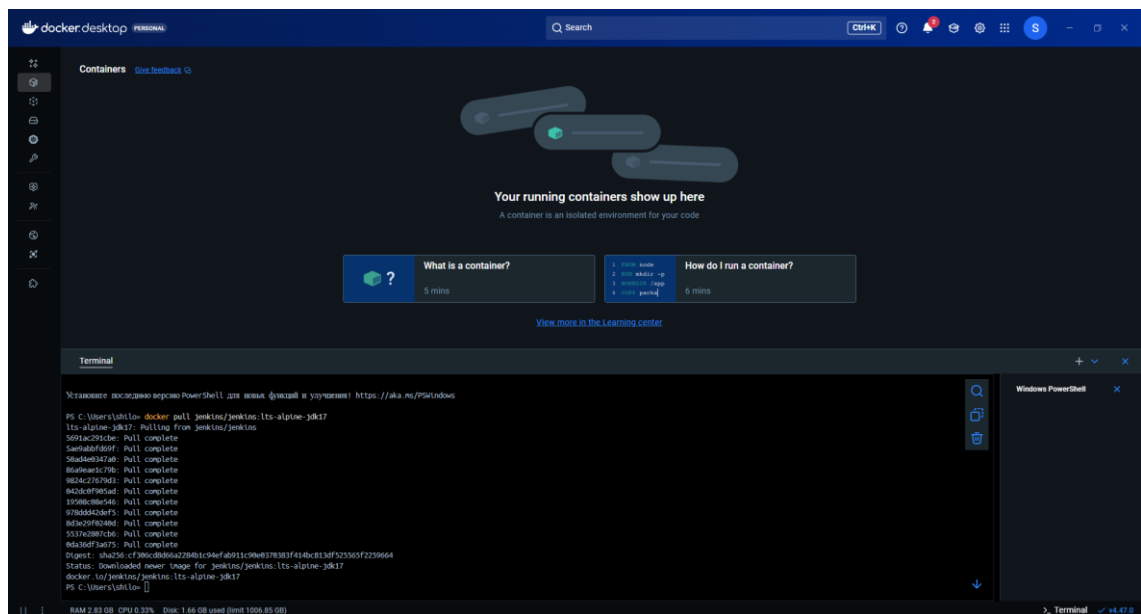


Рисунок 1 – Установка Jenkins

Запустим установленный образ, пробросим порты. На внешнем устройстве сайт будет доступен по порту 8080 (Рисунок 2).

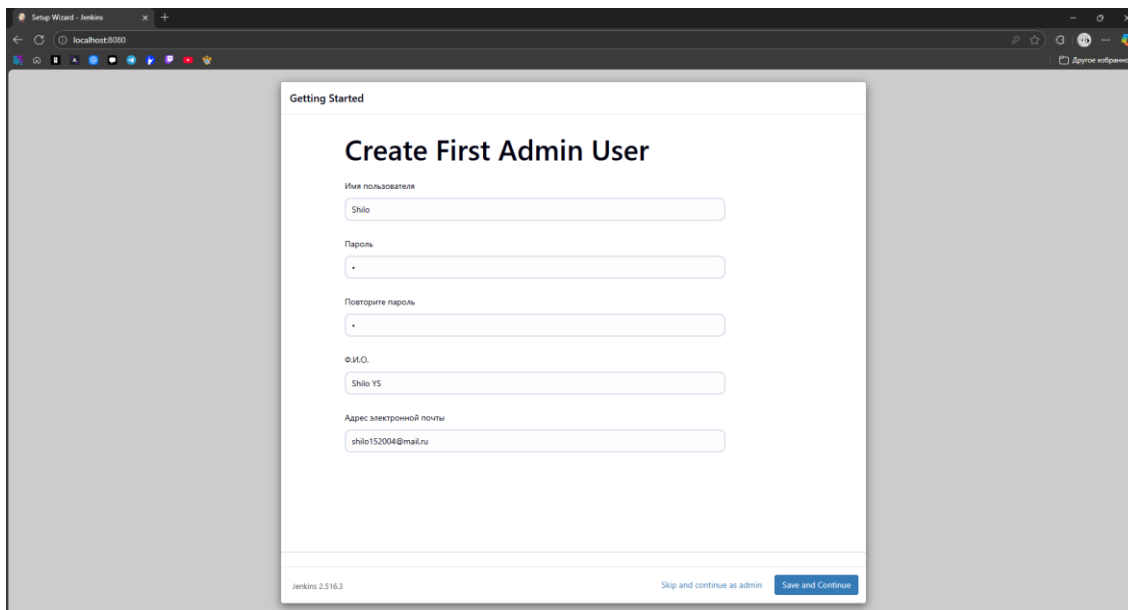


Рисунок 4 – Создание аккаунта администратора

После создания аккаунта администратора, нас перенаправит на главную страницу. Нажмем на “Create a job” для создания новой задачи. Выберем задачу со свободной конфигурацией, дадим ей название и описание. В настройках укажем, что данный для сборки будут братья из гитхаб репозитория (Рисунки 5, 6). Итоговой отображение таски показано на рисунке 7.

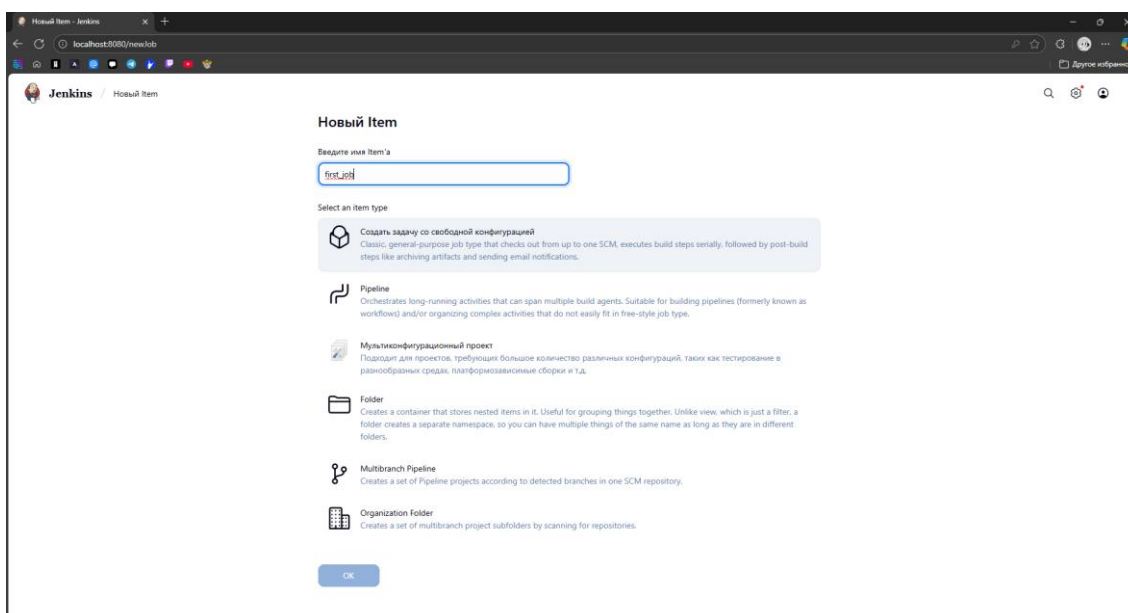


Рисунок 5 – Создание задачи со свободной конфигурацией

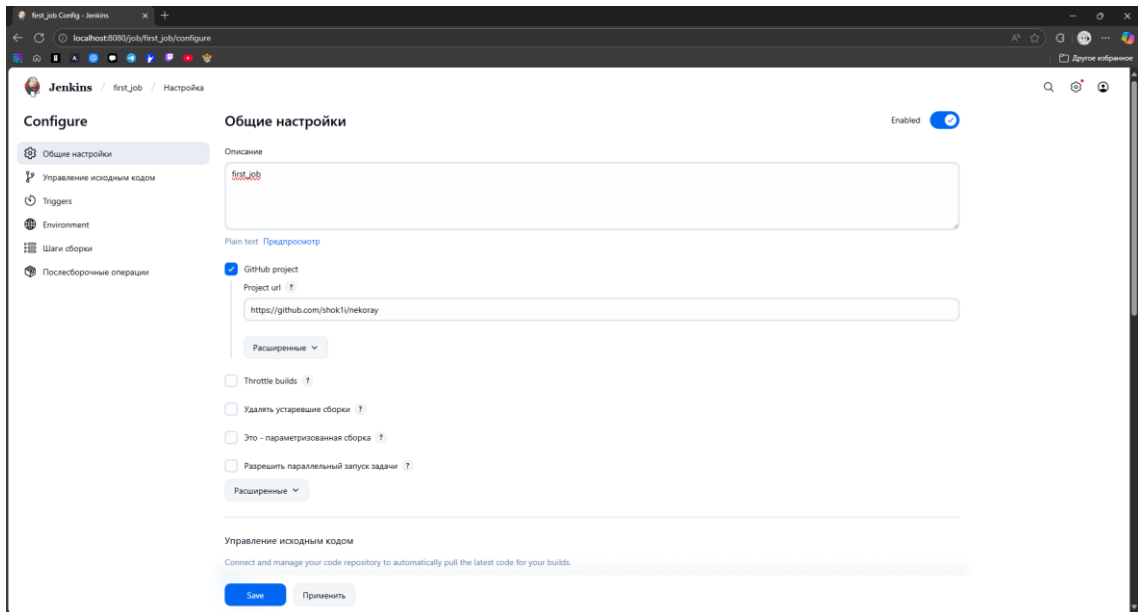


Рисунок 6 – Настройка задачи

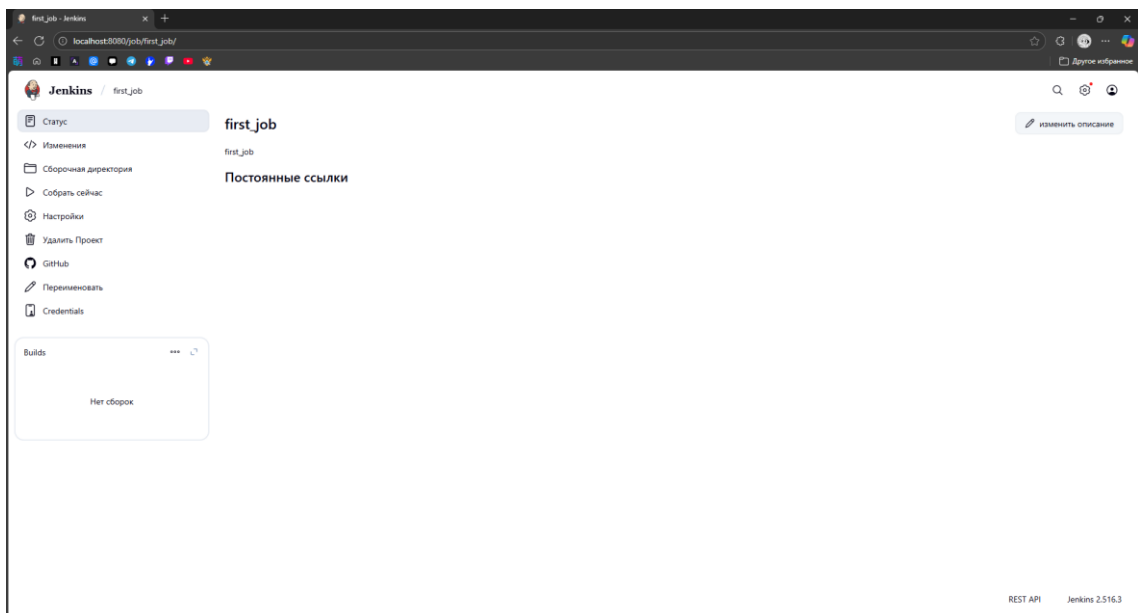


Рисунок 7 – Итоговое отображение таски

Попробуем собрать проект. Проект собрался успешно, что показано на рисунке 8.

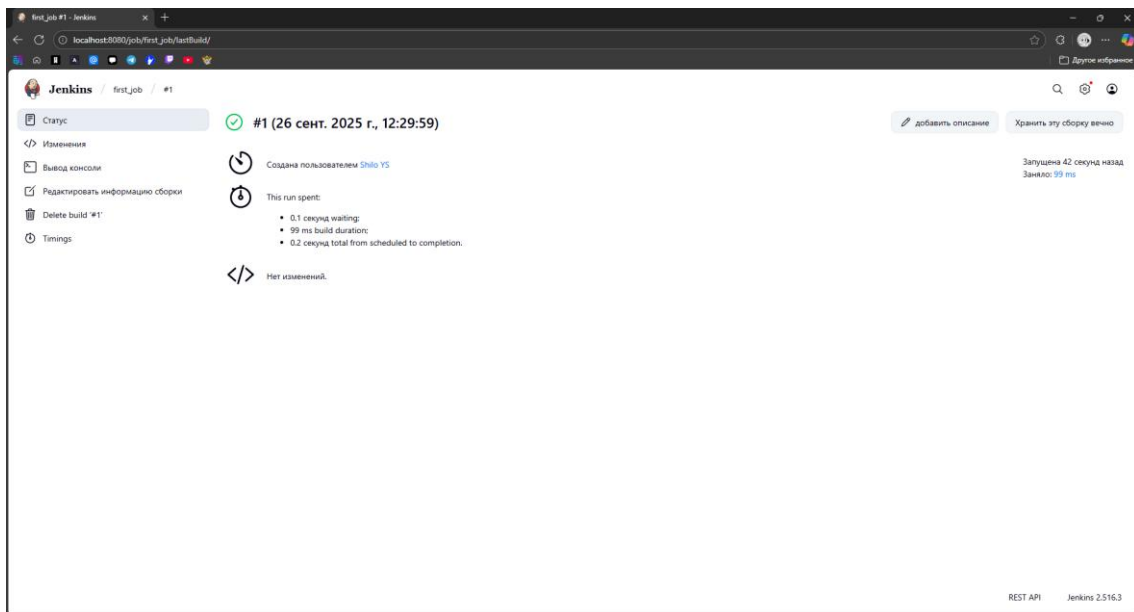


Рисунок 8 – Успешная сборка пустого проекта

Создадим pipeline job со скриптом, который будет выводить сообщения на каждом этапе работы (Рисунок 9).

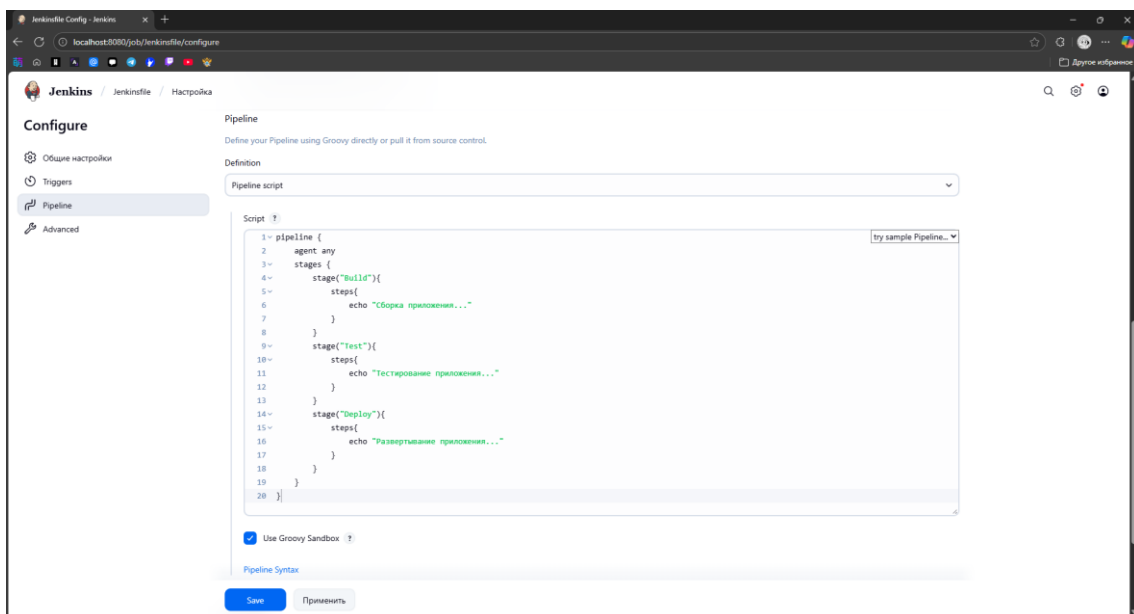


Рисунок 9 – Скрипт

Запустим сборку и проверим вывод консоли. Как можно заметить, выводимые с помощью команды “echo” надписи действительно появились в консоли (Рисунок 10).

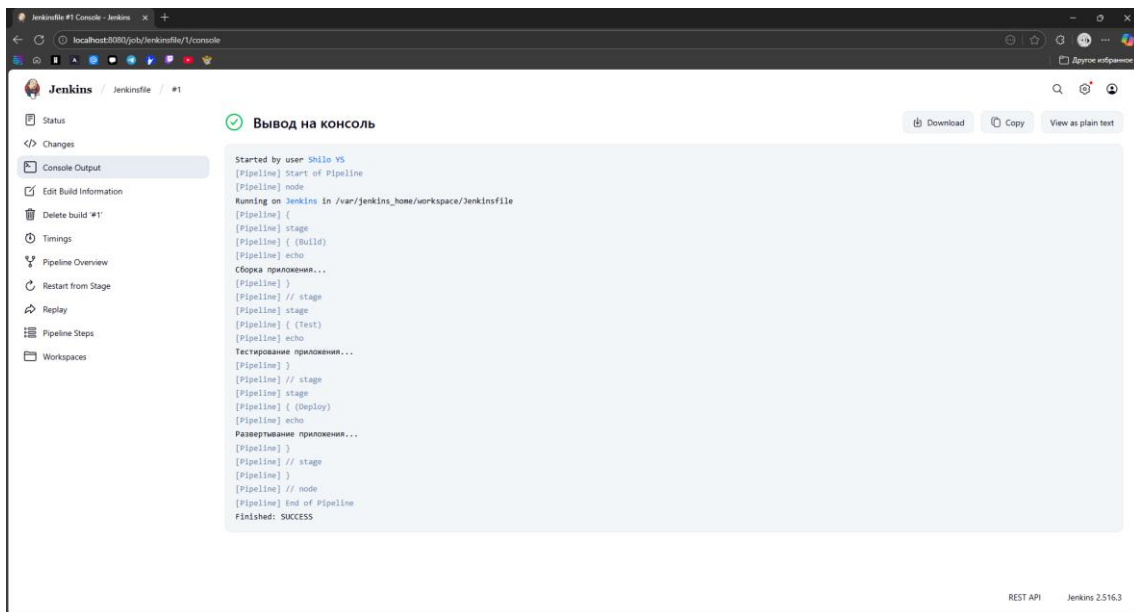


Рисунок 10 – Вывод консоли

Для симуляции работы с учетными данными создадим Credential (Рисунок 11).

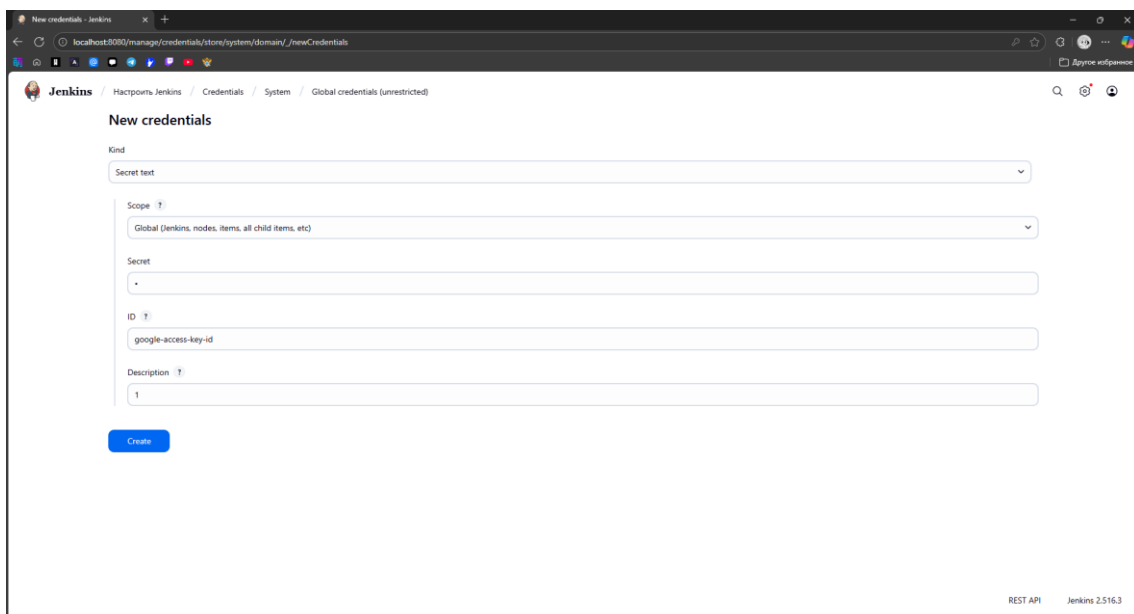


Рисунок 11 – Создание Credential

Настроим работу с отправкой писем. Для этого настроим раздел «Уведомление почтой», введя данные о SMTP сервере и почтовом адресе, с которого будут отправляться письма. Будем использовать специальный пароль, выдаваемый почтовым сервисом для работы с кодом (Рисунок 12).

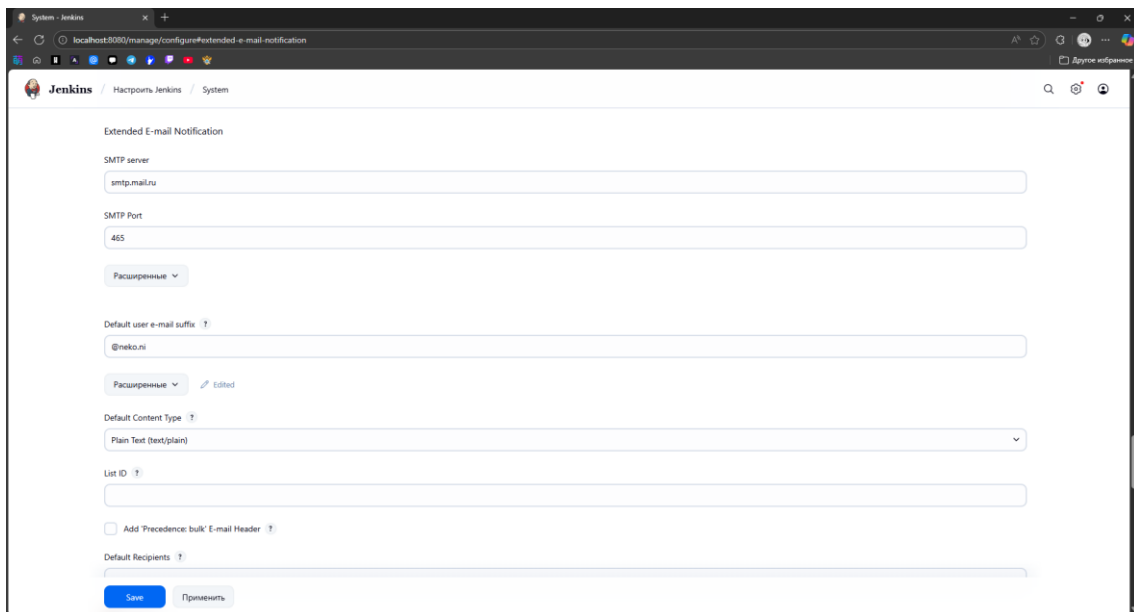


Рисунок 12 – Настройка почты

Также изменим скрипт в созданной job, добавив туда post. При неудачной сборке будет отправляться письмо на почту с сообщением о неудаче. Также добавим некорректную команду для работы с докером, чтобы «положить» сборку (Рисунок 13).

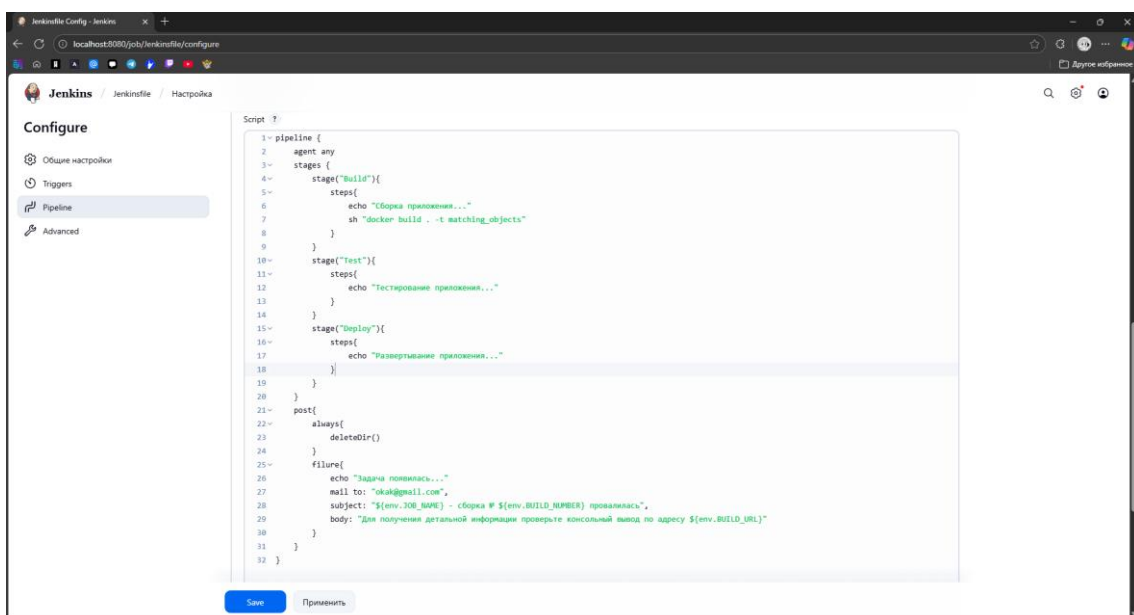


Рисунок 13 – Измененный для работы с почтой скрипт

Результат неудачной сборки можно наблюдать на рисунке 14.

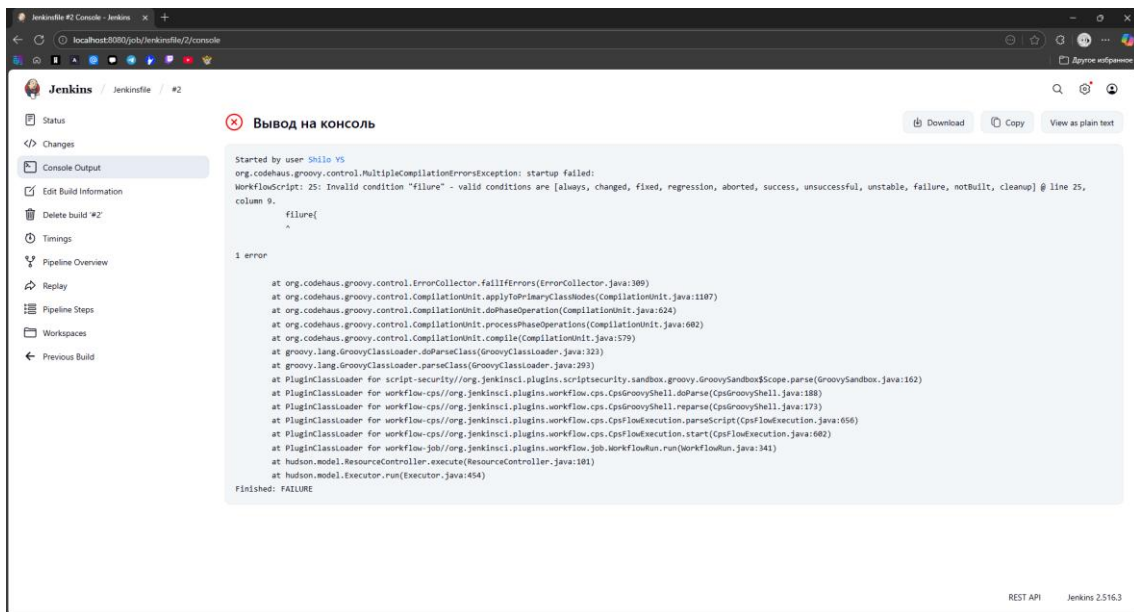


Рисунок 14 – Результат неудачной сборки проекта

Вывод

В ходе выполнения данной практической работы был создан Jenkins проект. С его помощью имитируется сборка проекта из github репозитория.

ПРАКТИЧЕСКАЯ РАБОТА 5

Задачи:

Руководствуясь данными теоретическими и лекционными материалами и исходя из выбранного проекта, вам необходимо изучить применение Lean-подходов для оптимизации процессов разработки программных продуктов на примере выбранного из списка вариантов задания приложения. Необходимо провести анализ текущего процесса с помощью картирования потока создания ценности (VSM), выявить узкие места, предложить Lean-решения и оценить их влияние. Для этого необходимо выполнить следующие задачи:

1. Дать краткое описание разрабатываемого приложения и той предметной области для которого делается данный программный продукт.

2. По аналогии с показанным выше подходом, провести анализ текущего состояния, представив карту текущего потока создания ценности (Current State VSM). Для этого опишите текущий процесс, определите ключевые этапы (сбор информации, анализ, разработка исправления, тестирование, развёртывание) и опишите как они организованы в текущем виде, выявите потери на каждом этапе, например, задержки из-за ручного тестирования, длительное согласование исправлений, перегруженность команды и т. д., ориентируясь, в том числе, на выбранную предметную область создаваемого приложения. Постройте диаграмму текущего состояния потока ценности (VSM).

3. Проанализировать виды потерь по Lean и определить основные проблемы, например, высокий объём незавершённых работ (WIP), длительное время исправления багов, перегруженность тестировщиков, избыточная документация.

4. Предложить Lean-решения для устранения потерь для конкретного проекта, например, автоматизация тестирования, внедрение Kanban-доски, CI/CD для ускорения релизов и т. д.

5. Построить диаграмму будущего состояния (Future State VSM), отражающую оптимизированный процесс потока создания ценности.

6. Рассчитать разницу во временных затратах до и после оптимизации. Оценить, какие метрики (время исправления, количество успешных релизов, плотность дефектов и т. д.) улучшились в результате внедрения Lean-подхода. Подбор требуемых метрик осуществляется самостоятельно.

Результат работы:

Проект направлен на оптимизацию процесса разработки мобильного приложения для настольных ролевых игр. Предметная область — создание кроссплатформенного мобильного приложения, включающего конструктор персонажей, базу данных правил, менеджер кампаний и интерактивные листы. Цель — сократить время вывода новых функций к пользователям, повысить качество продукта и снизить нагрузку на команду.

Диаграмма Current State VSM представлена на рисунке 1.

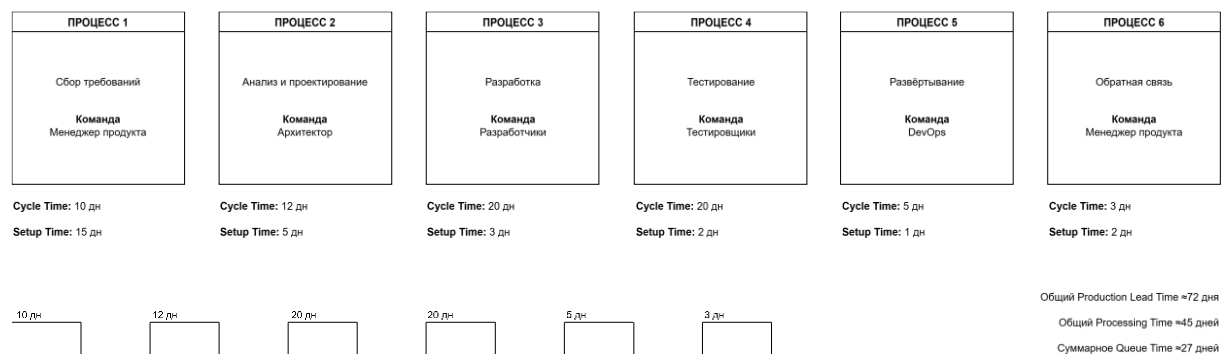


Рисунок 1. Current State VSM диаграмма

Этапы и проблемы:

1. Сбор информации
 - Менеджер продукта собирает крупные блоки требований, формирует ТЗ на целую версию.
 - Длительное ожидание финального утверждения ТЗ, неполные требования.
2. Анализ и разработка
 - Архитектор создает детальные спецификации и макеты для всего функционала.

- Разработчики работают одновременно над разными крупными модулями = высокий WIP и частые конфликты слияния.

3. Тестирование

- Проводится вручную по чек-листам после завершения всей разработки.

- Баги обнаруживаются поздно, большие очереди на тестирование.

4. Развёртывание

- Ручная сборка релизного APK/IPA и отправка в магазины приложений.

- Узкое место: длительный процесс модерации и риск человеческой ошибки.

Основные потери:

- Задержки из-за ручного тестирования и ожидания баг-репортов.
- Ручное развёртывание и сборка.
- Перегрузка тестировщиков в конце цикла.
- Высокий WIP и конфликты кода.
- Длительное согласование ТЗ и требований.

Виды потерь по Lean:

- Ожидание: задачи простаивают в очередях на тестирование и утверждение.

- Избыточные действия: ручное тестирование, ручной деплой, избыточная документация.

- Перегрузка сотрудников: тестировщики в конце цикла, разработчики при решении конфликтов.

- WIP: несколько крупных задач одновременно у разработчиков.

- Дефекты: баги выявляются поздно из-за отсутствия ранних авто тестов.

Lean-решения:

- Внедрение Kanban-доски для визуализации потока и ограничения WIP.
- Автоматизация тестирования (юнит-тесты, UI-тесты через Appium).
- Внедрение CI/CD конвейера (GitHub Actions/GitLab CI) для автоматической сборки, тестирования и деплоя.
- Переход на короткие итерации (спринты по 1-2 недели) и разбиение фич на мелкие истории.
- Настройка Code Review + Pull Requests для снижения дефектов.

В результате была построена диаграмма Future State VSM, представленная на рисунке 2.

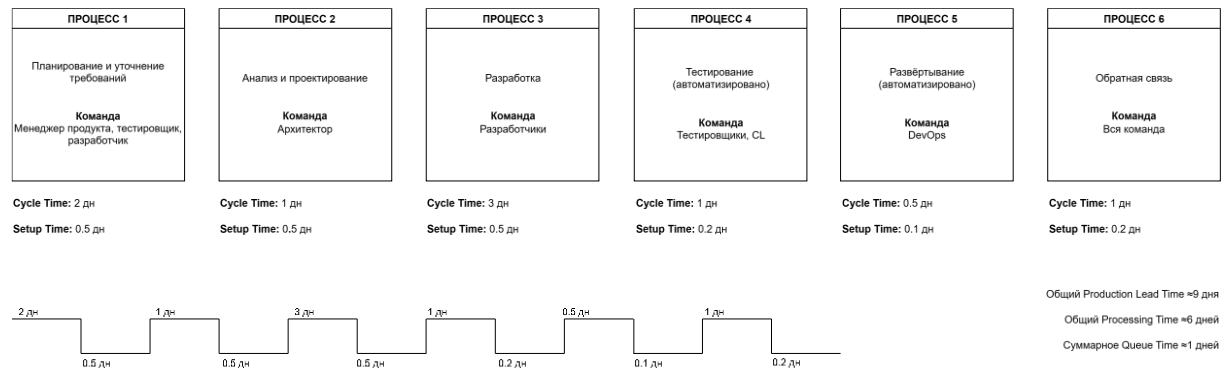
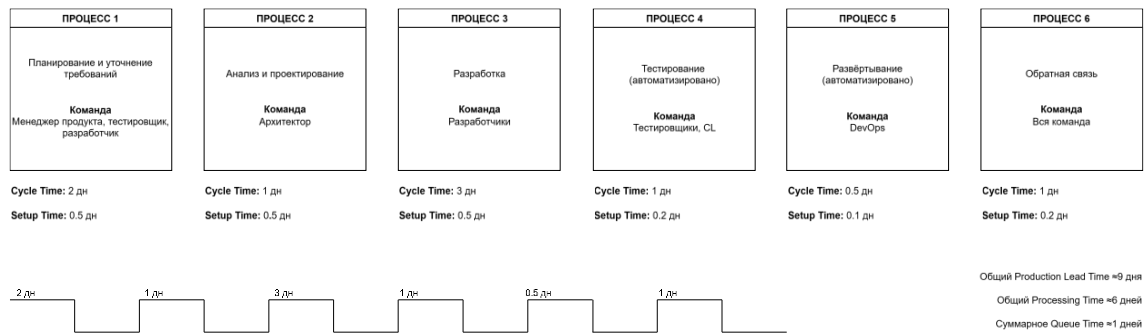


Рисунок 2. Диаграмма Future State VSM

До (Current State):

- Среднее время изменения (Lead Time): ~72 дня
- Ручное тестирование: ~20 дней
- Деплой: ~5 дней
- Частота релизов: 4-5 раз в год
- Плотность дефектов: высокая (20+ багов за релиз)



Lead Time: ~8-9 дней (сокращение на 87.5%)

- Тестирование автоматизировано → минус 18.5 дней
- Деплой автоматизирован → минус 4 дня
- Частота релизов: несколько раз в месяц
- Плотность дефектов: ниже на ~75-90%
- Деплой автоматизирован → минус 0,5 дня.
- Частота релизов: несколько раз в неделю.
- Плотность дефектов: ниже на ~30–40%.

Вывод

В ходе работы был проведён анализ текущего процесса разработки и доставки микросервисов с помощью VSM (Value Stream Mapping). Построена карта текущего состояния, выявлены основные узкие места. Проведена классификация потерь по Lean и предложены решения.

ПРАКТИЧЕСКАЯ РАБОТА 6

Задачи:

Руководствуясь данными теоретическими и лекционными материалами и исходя из выбранного проекта, вам необходимо изучить применение методологии Lean Startup и Design Thinking для проверки гипотез, быстрого прототипирования, получения обратной связи и инновационного подхода к решению проблемы, на примере выбранного из списка вариантов задания стартапа. Для этого необходимо выполнить следующие задачи:

1. Сформулировать основные задачи стартапа, выявить проблемы целевой аудитории и понять потребности пользователей. Для этого проанализировать существующие решения на рынке и выявить их слабые стороны, по возможности, найти различные интервью и опросы с пользователями, малым бизнесом и т. д. для выявления текущих проблем, характерных для выбранной предметной области. Оценить общие технологические тренды (например, повышение производительности мобильных устройств) и потребности будущих пользователей (например, рост использования пользователями мобильных устройств).

2. Исходя из данного анализа, сформулировать чёткую проблему стартапа.

3. Сформулировать гипотезу стартапа. Уделить повышенное внимание важности правильного определения гипотезы и механизмов дальнейшей его проверки в рамках стартапа.

4. Придумать несколько возможных простых идей для инновационных функций выбранного приложения.

5. Описать базовый и инновационный функционал MVP (минимально жизнеспособный продукт).

6. Используя инструменты для раннего прототипирования (например ProtoPie в связке с Figma с Starter планом) создать интерактивный прототип, включающий ключевые функции MVP. Прототип должен быть простым и

интуитивно понятным, чтобы минимизировать затраты времени и ресурсов. Реализовывать MVP на соответствующем языке программирования не нужно.

7. Эмулировать тестирование прототипа с реальными пользователями (другими обучающимися). Для этого провести интервью с пользователями, чтобы узнать их мнение о функционале, интерфейсе и удобстве использования. Собрать данные о том, что нравится пользователям, а что нужно улучшить. Здесь можно использовать A/B тестирование для проверки различных вариантов интерфейса. Определить какие изменения будут вноситься в прототип на основе полученной обратной связи.

8. Внести изменения в прототип приложения, основываясь на полученной обратной связи. Оценить результаты после внесения изменений и принять решение о дальнейшем развитии функционала.

Результат работы:

Основные задачи стартапа:

- Создать единый календарь релизов игр с возможностью фильтрации по платформе, жанру, издателю и дате выхода.
- Добавить систему, которая показывает примерную длительность прохождения.
- Позволить пользователям вести личный список игр: «ожидая», «прохожу», «завершил».
- Обеспечить социальный компонент — обмен отзывами, рекомендациями и статусами друзей.

Проблемы целевой аудитории (геймеры 16–35 лет):

- Нет единого приложения, где удобно следить за будущими релизами и длительностью прохождения игр одновременно.
- HowLongToBeat не показывает календарь релизов, а лишь базу по длительности.
- Крупные порталы не дают персонализированных уведомлений о выходе игр.

- Геймеры часто пропускают интересные проекты или забывают о релизах, которые добавляли «в желания».
- Нет простого способа синхронизировать личный список игр с друзьями.

Анализ существующих решений:

- HowLongToBeat — сильная база данных длительности, но нет календаря и уведомлений.
- Metacritic / IGDB / Steam — дают информацию о датах, но нет социальной составляющей.
- GameTrack / GG / Backloggd — фокус на учёте пройденных игр, но не на предстоящих релизах.

Слабые стороны существующих решений:

- Отсутствие функции календаря и напоминаний о релизах.
- Сложная или устаревшая навигация.
- Нет интеграции с рекомендациями по времени.
- Отсутствие геймификации и социального взаимодействия.

Технологические тренды:

- Рост популярности мобильных приложений для планирования досуга и медиа-потребления.
- Геймификация пользовательского опыта (ачивки, уровни, бейджи).
- Популярность микросообществ и социальных фидов вокруг интересов.

Потребности будущих пользователей:

- Удобный календарь игровых релизов с уведомлениями.
- Возможность оценить, сколько времени займёт прохождение.
- Функция «в ожидании» / «прохожу» / «завершил».

- Рекомендации по играм, подходящим под свободное время.
- Геймификация.

Чёткая проблема стартапа - Геймерам сложно следить за релизами, оценивать, сколько времени займёт прохождение игр, и планировать свой игровой досуг. Существующие платформы либо информируют, либо фиксируют прогресс, но не объединяют эти функции в одном месте.

Гипотеза стартапа - Если создать мобильное приложение, которое объединяет календарь релизов, данные о длительности прохождения и социальные функции, пользователи смогут лучше планировать игровое время, открывать новые игры и обмениваться рекомендациями.

Идеи для инновационных функций:

- Интеграция с базой данных HowLongToBeat API или собственной системой расчёта длительности.
- Интеллектуальные рекомендации: «Игры, которые можно пройти за выходные» и т.п.
- Уведомления о релизах игр из списка ожидания.
- Социальная лента — статус друзей.
- Геймификация профиля: уровни, бейджи «Игроман месяца», «Скоростной проходчик» и т. п.

MVP:

Базовый функционал:

- Календарь с релизами игр по датам.
- Профиль пользователя с личным списком игр.
- Фильтры по жанрам, платформам, издателям.
- Поиск игр.

Инновационный функционал:

- Интеграция данных о длительности прохождения.

- Система списков: «ожидая», «прохожу», «завершил».
- Напоминания о релизах.
- Простейшие социальные элементы.

Прототип:

Изображение схематики прототипа представлено на рисунке 1.

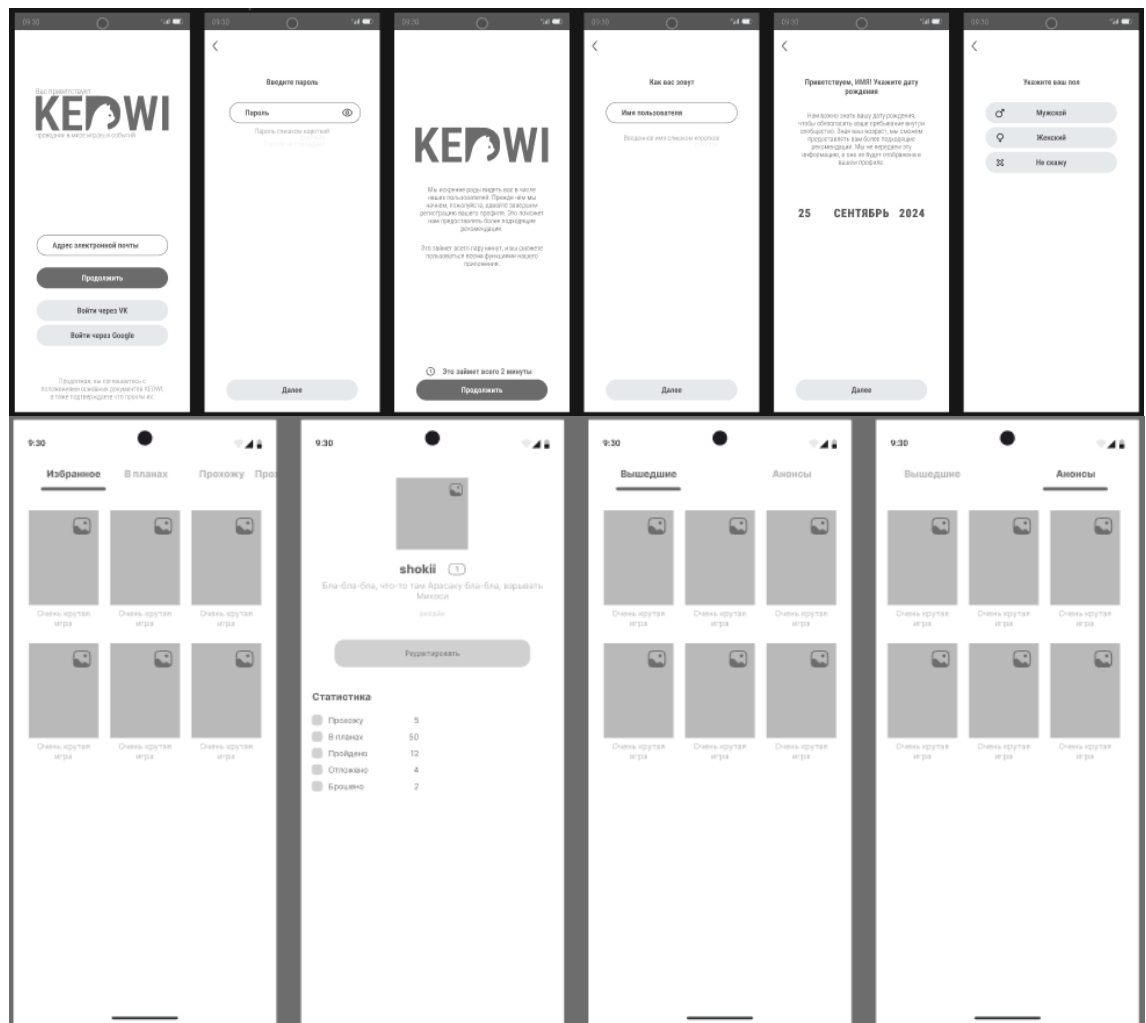


Рисунок 1. Wireframe приложения

Результаты тестирования прототипа:

- 5 из 6 участников отметили, что календарь релизов с длительностью прохождения «помогает планировать время».
- 4 из 6 понравилась идея «списков игр» как личного дневника.
- 3 из 6 выразили интерес к уведомлениям о релизах и рекомендациям по времени.

Внесённые изменения:

- Добавлена функция уведомлений с выбором частоты.
- В карточку игры добавлен блок «Подобные по длительности».
- Улучшен интерфейс фильтров по жанрам и платформам.

Вывод

В ходе работы были применены принципы Lean Startup и Design Thinking для создания концепции мобильного приложения — календаря игровых новинок. Определены задачи стартапа, выявлены потребности пользователей и недостатки аналогов. Сформулирована проблема и гипотеза, предложены инновационные функции и описан MVP. После тестирования прототипа внесены улучшения, направленные на повышение удобства планирования и персонализации игрового опыта.

ПРАКТИЧЕСКАЯ РАБОТА 7

Цель работы

Целью данной практической работы является освоение принципов методологии Knowledge-Centered Service (KCS), направленной на создание и поддержку актуальной базы знаний в процессе разработки и сопровождения программного обеспечения. Дополнительно будет рассмотрен вопрос совместного использования методологии Knowledge-Centered Service (KCS) с другими методологиями, практиками и решениями. В ходе выполнения работы студенты научатся документировать и структурировать знания, возникающие в процессе решения задач проекта, а также формировать и обслуживать реализованную систему знаний на базе открытых инструментов.

Задание на самостоятельное выполнение

Руководствуясь данными теоретическими и лекционными материалами и исходя из выбранного проекта, вам необходимо изучить применение методологии управления знаниями KCS для создания базы знаний в процессе командной разработки программного обеспечения, на примере выбранного из списка вариантов задания ситуации. Требуется реализовать автоматическую генерацию статей в BookStack на основе новых задач (Issue) в репозитории GitHub, используя GitHub Actions и API-интерфейс BookStack. Для этого необходимо выполнить следующие задачи:

1. Дать подробное описание разрабатываемого командой программного продукта, в том числе предполагаемый к использованию стек и уже существующую ИТ инфраструктуру, используемой компанией. Также описать саму структуру команды и решаемую конкретную ситуацию (в примере была рассмотрена ситуация с организацией командой базы знаний технической поддержки приложения для онлайн-курсов).
2. Установить и запустить BookStack, например при помощи Docker.
3. Сделать BookStack доступным в сети, например, с использованием решений, показанных в данном методическом пособии.

4. Выполнить первоначальную настройку BookStack, а именно создать API пользователя и сгенерировать API-токен.

5. Создать шаблон статьи, соответствующий методологии KCS. Он должен иметь некоторое стандартное наполнение и включать следующие разделы:

- Проблема
- Причина
- Решение
- Ссылки, в том числе на Issue на GitHub
- Метки

6. Сохранить шаблон и использовать его при создании новых задач (Issue) в GitHub.

7. Создать репозиторий и настроить GitHub Secrets.

8. Разработать GitHub Action, с функционалом обработки события issues: opened, а именно извлечением заголовка и тела Issue, конвертации Markdown в HTML с использованием библиотеки marked, и отправки POST-запроса к API BookStack с созданием страницы. Дополнительно требуется реализовать автоматическое обновление статьи, если задание редактируется (issues.edited) или закрывается (issues.closed).

9. Продемонстрировать работоспособность полученного решения, на примере выбранной ситуации (выбранного варианта задания).

10. Подготовить отчёт с подробным описанием всех выполненных шагов.

Результат работы

Разработан Telegram-бот для интеграции с корпоративной базой знаний компании «EduPortal». Решение предоставляет сотрудникам службы поддержки мгновенный доступ к актуальным инструкциям, возможность создания новых статей через GitHub Issues и получение уведомлений об изменениях документации.

Стек:

- Telegram Bot API — основа для взаимодействия с пользователями в мессенджере;
- BookStack — платформа для хранения и организации документации;
- Docker — контейнеризация и развёртывание всех компонентов системы;
- GitHub — хранение репозитория с шаблонами задач;
- GitHub Actions — автоматизация обработки событий Issues;
- Node.js с библиотекой marked — преобразование Markdown в HTML для публикации в BookStack.

Состав команды:

- Тимлид — проектирование архитектуры и интеграционных решений;
- 2 разработчика бэкенда — настройка Docker-окружения, GitHub Actions и функционала бота;
- DevOps-инженер — развёртывание BookStack и организация сетевого доступа;
- Технический писатель — разработка шаблонов и написание контента для базы знаний.

Решаемая ситуация:

Сотрудники поддержки тратят значительное время на поиск необходимой информации. Внедрение Telegram-бота обеспечивает оперативный доступ к решениям, упрощает процесс создания новых статей через GitHub и способствует повышению эффективности работы команды при снижении нагрузки на службу поддержки.

Для установки bookstack опишем docker-compose.yml файл. Содержимое файла docker-compose.yml предоставлено ниже.

```
volumes:
  db: {}

services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: bookstack-dev
      MYSQL_USER: bookstack-test
      MYSQL_PASSWORD: bookstack-test
      MYSQL_RANDOM_ROOT_PASSWORD: 'true'
    volumes:
      - ./dev/docker/init.db:/docker-entrypoint-initdb.d
      - db:/var/lib/mysql
  app:
    build:
      context: .
      dockerfile: ./dev/docker/Dockerfile
    environment:
      APP_URL: http://192.168.1.102:${DEV_PORT:-8080}
      DB_CONNECTION: mysql
      DB_HOST: db
      DB_PORT: 3306
      DB_DATABASE: bookstack-dev
      DB_USERNAME: bookstack-test
      DB_PASSWORD: bookstack-test
      TEST_DATABASE_URL: mysql://bookstack-test:bookstack-test@db/bookstack-test
      MAIL_DRIVER: setup
      MAIL_HOST: mailing
      MAIL_PORT: 1025
    ports:
      - ${DEV_PORT:-8080}:80
    volumes:
      - ./:/app
      - ./dev/docker/php/conf.d/xdebug.ini:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
    entrypoint: /app/dev/docker/entrypoint.app.sh
    extra_hosts:
      - "host.docker.internal:host-gateway"
  node:
    image: node:22-alpine
    working_dir: /app
    user: node
    volumes:
      - ./:/app
    entrypoint: /app/dev/docker/entrypoint.node.sh
  mailing:
    image: maildev/maildev
    ports:
      - ${DEV_MAIL_PORT:-8025}:8025
```

Успешный запуск докер контейнера предоставлен на рисунке 1.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4752b96c62a8	bookstack_app	"/app/dev/docker/ent..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	bookstack_app_1
f5248eb66672	node:22-alpine	"/app/dev/docker/ent..."	About a minute ago	Up About a minute		bookstack_node_1
e6d3d15d6d83	mysql:8.0	"docker-entrypoint.s..."	About a minute ago	Up About a minute	3306/tcp, 33066/tcp	bookstack_db_1
71fc98dd4421	mailhog/mailhog	"MailHog"	About a minute ago	Up About a minute	1025/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8025/tcp	bookstack_mailhog_1

Рисунок 1 – Описание запущенных контейнеров

Перейдем на веб интерфейс BookStack. Веб интерфейс успешно запустился, скриншот демонстрации предоставлен на рисунке 2.

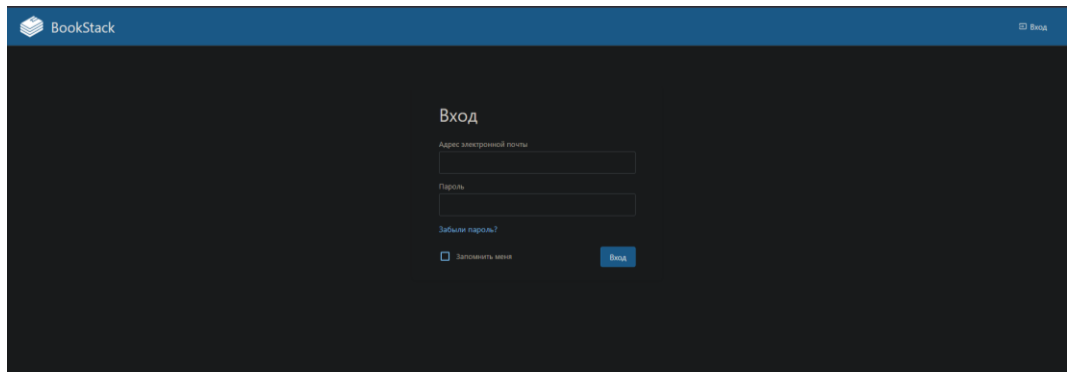


Рисунок 2 – Запущенный BookStack

Для дальнейшего выполнения задания добавим пользователя. Раздел для создания пользователя представлен на рисунке 3.

Add New User

User Details

Set a display name and an email address for this user. The email address will be used for logging into the application.

Name

Email

▶ External Authentication ID

User Roles

Select which roles this user will be assigned to. If a user is assigned to multiple roles the permissions from those roles will stack and they will receive all abilities of the assigned roles.

☒ Admin ☐ Public

☐ Editor ☐ Viewer

User Password

You can choose to send this user an invitation email which allows them to set their own password otherwise you can set their password yourself.

☒ Send user invite email

Preferred Language

This option will change the language used for the user-interface of the application. This will not affect any user-created content.

English ▼

Рисунок 3 – Интерфейс создания пользователя

Для созданного пользователя создадим свой API-Token. Успешное создание API-Token представлено на рисунке 4.

API Token

Token ID
This is a non-editable system generated identifier for this token which will need to be provided in API requests.
PsSefQj509Q7iBiPkV8M2kY3Tm9CC3UE

Token Secret
This is a system generated secret for this token which will need to be provided in API requests. This will only be displayed this one time so copy this value to somewhere safe and secure.
ppuds2z2f82qeRjbKGWEzygHxnzVn6xG

Name
Give your token a readable name as a future reminder of its intended purpose.
main-api-token

Expiry Date
Set a date at which this token expires. After this date, requests made using this token will no longer work. Leaving this field blank will set an expiry 100 years into the future.
15.02.4444

Token created 0 seconds ago
Token updated 0 seconds ago

Back Delete Token Save

Рисунок 4 – Успешное создание API-Token для пользователя

Создадим статью шаблон для последующего использования. Описание шаблона представлено на рисунке 5.

```
GNU nano 6.2
name: KCS-шаблон задачи
about: Задокументировать инцидент или важное знание для базы знаний
title: "[KCS]"
labels: ["knowledge", "auto-export"]
assignees: ''

# Краткое описание проблемы
<!-- Опишите проблему в 1-2 предложениях -->

## Проблема
<!-- Подробно опишите, что произошло, при каких условиях возникает ошибка или ситуация -->

## Причина
<!-- Укажите первопричину возникновения проблемы (если известна, иначе оставить пустым для анализа) -->

## Решение
<!-- Как проблему можно решить или временные обходные пути -->

## Шаги для воспроизведения
<!-- Пример:
1. Зайти на страницу авторизации
2. Ввести неверный пароль
3. Нажать "Войти"
-->

## Ожидаемый результат
<!-- Что должно происходить -->

## Фактический результат
<!-- Что происходит на самом деле -->

## Ссылки и примечания
- Ссылка на связанный коммит/PR:
- Дополнительные материалы или скриншоты

## Метки
<!-- Укажите ключевые теги/категории: bug, feature, urgent, frontend, backend и т.д. -->
```

Рисунок 5 – Файл с описанием шаблона статьи

Теперь данный шаблон можно использовать в заполнении Issues на GitHub. Демонстрация применения шаблона на рисунке 6.

Create new issue in KopIAkash1/bookstack: KCS-шаблон задачи

Add a title *

[KCS]

Add a description

Write Preview

Краткое описание проблемы
<!-- Опишите проблему в 1-2 предложениях -->

Проблема
<!-- Подробно опишите, что произошло, при каких условиях возникает ошибка или ситуация -->

Причина
<!-- Укажите первопричину возникновения проблемы (если известна, иначе оставить пустым для анализа) -->

Решение
<!-- Как проблему можно решить или временные обходные пути -->

Шаги для воспроизведения
<!-- Пример:
1. Зайти на страницу авторизации
2. Ввести неверный пароль
3. Нажать "Войти"
-->

Ожидаемый результат
<!-- Что должно происходить -->

Фактический результат
<!-- Что происходит на самом деле -->

Ссылки и примечания

☐ Create more Cancel Create

Рисунок 6 – Демонстрация шаблона на GitHub

Далее для взаимодействия по API настроим секреты в GitHub.

Скриншот завершенной настройки представлен на рисунке 7.

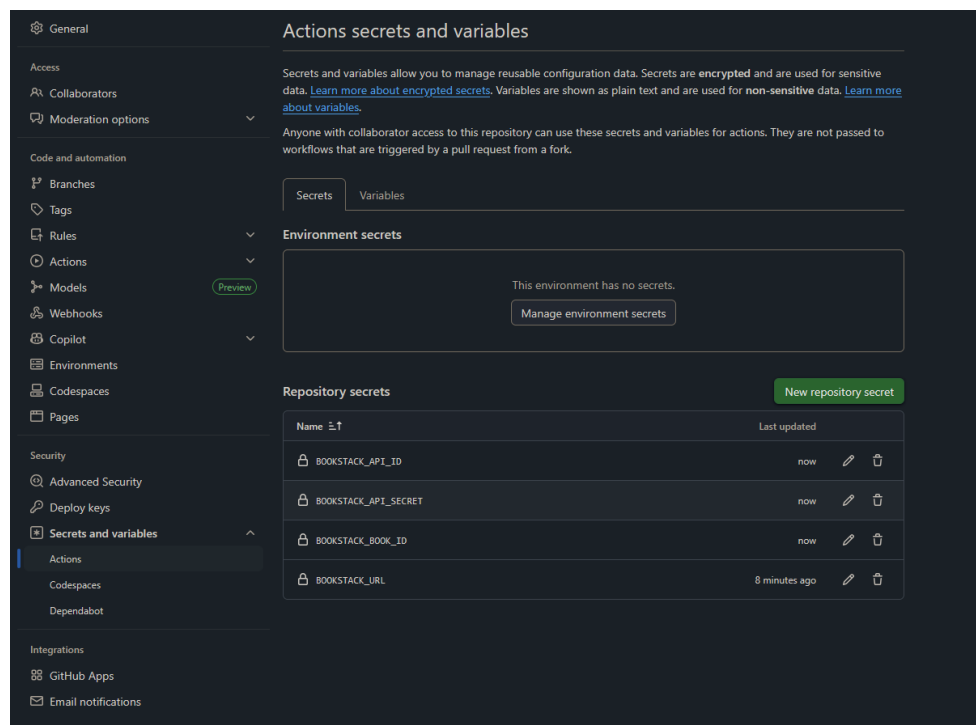


Рисунок 7 – Настроенные github secrets

Опишем пайплайн для добавления issue в страницы. Результат представлен на рисунке 8.

```

Code Blame 32 lines (27 loc) · 972 Bytes
1  name: Create BookStack Page from Issue
2
3  on:
4    issues:
5      types: [opened, edited, closed]
6
7  jobs:
8    create-page:
9      runs-on: ubuntu-latest
10     steps:
11       - name: Checkout repository
12         uses: actions/checkout@v3
13
14       - name: Setup Node.js
15         uses: actions/setup-node@v3
16         with:
17           node-version: 20
18
19       - name: Install dependencies
20         run: npm install marked
21
22       - name: Run BookStack script
23         env:
24           BOOKSTACK_URL: ${ secrets.BOOKSTACK_URL }
25           BOOKSTACK_BOOK_ID: ${ secrets.BOOKSTACK_BOOK_ID }
26           BOOKSTACK_API_ID: ${ secrets.BOOKSTACK_API_ID }
27           BOOKSTACK_API_SECRET: ${ secrets.BOOKSTACK_API_SECRET }
28           ISSUE_TITLE: ${ github.event.issue.title }
29           ISSUE_BODY: ${ github.event.issue.body }
30           ISSUE_URL: ${ github.event.issue.html_url }
31           ISSUE_LABELS: ${ join(github.event.issue.labels.*.name, ' ') }
32         run: node .github/scripts/bookstack.js

```

Рисунок 8 – Пайплайн добавления issue в bookstack

Далее создадим Issue с описанием, проверим выполнение пайплайна. Оба этих пункта отображены на рисунке 10.

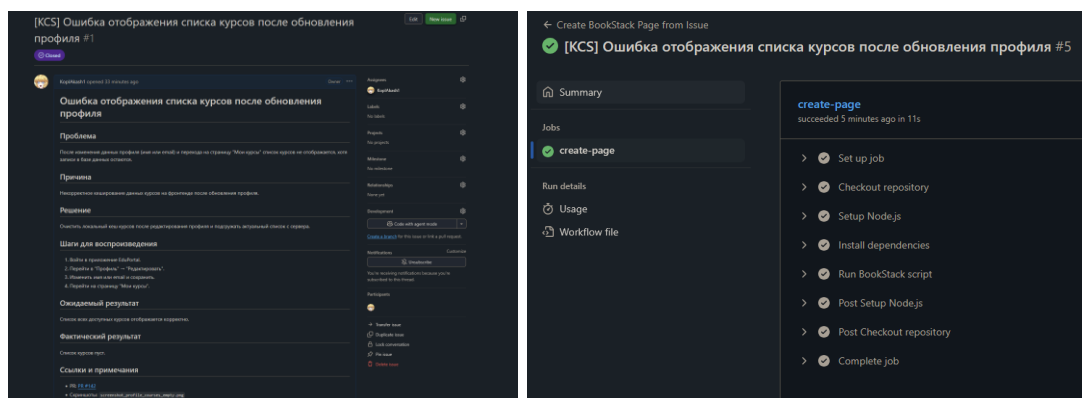


Рисунок 10. Созданный Issue и успешно отработавший пайплайн

Вывод

В ходе работы была реализована автоматизация создания базы знаний по методологии KCS для командной разработки. Настроен BookStack с доступом через API, разработан шаблон статьи KCS и скрипт на Node.js, который формирует HTML-страницы из данных GitHub Issues. С помощью GitHub Actions обеспечена автоматическая генерация страниц при создании, редактировании и закрытии задач.